EcommerceHub Backend

COMPREHENSIVE ARCHITECTURAL REPORT & COMPLETE SOURCE REGISTRY

Repository URL: https://github.com/manoharmk/EcommerceHub-E-commerce-Backend-Application

Target Branch: main

Author Identity: manoharmk <manuken123@gmail.com>

Database Platform: PostgreSQL 16.2 on Port 5432 (Database name: myapp)

Build System: Maven & JDK 17 (Eclipse Temurin)

Report Date: June 2026

Part 1: System Architecture & Layered Design

The EcommerceHub backend is designed using a clean, decoupling-focused layered MVC (Model-View-Controller) pattern built over Spring Boot 3.5.x and Hibernate JPA, backed by PostgreSQL. The system enforces unidirectional flow of data and isolates external APIs from the database schema.

1. Controller Layer: Receives HTTP REST requests, performs basic schema/format validations using JSR-380 annotations (@Valid, @NotBlank, etc.), and maps payloads into service calls. Returns JSON responses wrapped in standard ResponseEntity.

2. Service Layer: Orchestrates business rules, state transitions, security filters, stock check-outs, and mapping details. This is where transactional boundaries are managed via @Transactional.

3. Repository Layer: Interfaces with PostgreSQL using Spring Data JPA. By extending JpaRepository, the system avoids manual boilerplate SQL queries. Dynamic queries are generated at runtime by naming patterns (e.g. findByCartIdAndProductId).

4. Model / Entity Layer: Relational models configured using Hibernate JPA annotations. Defines the tables, primary keys, foreign keys, and relational cardinality (such as One-to-Many and Many-to-Many relations).

5. DTO / Payload Layer: Eliminates data over-exposure. Entities are mapped to DTOs (Data Transfer Objects) using ModelMapper before sending them to the client. This blocks accidental write-backs or circular JSON parsing errors.

Part 2: Database Schema & Entity Relationships

The application maps entity tables in PostgreSQL. The system uses a clean relational structure:

- users: Primary identity table containing credentials and user roles. Linked to orders, addresses, and carts.
- roles: Pre-seeded enumeration of access levels (ROLE_USER, ROLE_SELLER, ROLE_ADMIN).
- user_role: Join table mapping user accounts to permissions.
- categories: Logical grouping of products (e.g. Electronics, Books).
- products: The catalog table. Includes product names, images, descriptions, quantities, discounts, base prices, and special discounted prices.
- carts: The active shopping containers. Maps to a single User.
- cart_items: Join container linking products to carts. Includes dynamic details (quantity, price, specialPrice).

- orders: Persistent checkout snapshots. Maps to addresses, users, payments, and order items.
- order_items: Relational log capturing the exact pricing and quantities at the time of order completion.
- payments: Captures checkout transaction parameters (payment methods, confirmation details).
- addresses: Multi-entry user addresses for delivery details.

Part 3: Stateless Authentication & Security

Authentication is implemented using a stateless, cookie-based Spring Security architecture utilizing JSON Web Tokens (JWT):

1. Login Interception: Users log in using username and password via AuthController. The password is compared to the database's BCrypt hash.

2. HttpOnly Cookie Storage: Upon verification, a JWT is generated. The token is stored inside an HttpOnly cookie named 'springBootEcom'. This cookie is marked secure and is invisible to client JavaScript, providing defense against Cross-Site Scripting (XSS) attacks.

3. Authentication Filters: Every incoming request passes through the AuthTokenFilter. The filter parses the request cookies, extracts the 'springBootEcom' JWT, verifies the signature using the HS512 secret key, extracts the active username, loads details using UserDetailsService, and builds a UsernamePasswordAuthenticationToken which is placed in the SecurityContextHolder.

Part 4: Key API Workflows & Concrete Logic Examples

A. User Signup & SignIn Flow

Registration Flow:
1. Client POSTs details to '/api/auth/signup'.
2. AuthController checks UserRepository to verify username and email uniqueness.
3. User entity is saved with password hashed via BCryptPasswordEncoder.
4. Standard user role (ROLE_USER) is attached and saved.

SignIn Flow:
1. Client POSTs credentials to '/api/auth/signin'.
2. AuthenticationManager validates credentials against DB hashed values.
3. JwtUtils compiles user details and signs a new token with the application's secret key.
4. The token is appended to the Response's 'Set-Cookie' header. Client receives a UserInfoResponse JSON payload.

B. Add Product to Cart Flow

1. Client makes POST to '/api/carts/products/{productId}/quantity/{quantity}'.
2. CartController extracts the active authenticated user's email via security context.
3. CartServiceImpl fetches the user's cart (or initializes a clean one if not found).
4. ProductRepository retrieves the target product. System checks if (product.quantity >= quantity).
5. CartItemRepository checks if the product is already in the cart. If yes, quantity is incremented. If no, a new CartItem is created.
6. Special price is dynamically calculated: price - (price * discount / 100).
7. Cart's totalPrice is updated, changes are flushed, and a clean CartDTO is returned.

C. Order Placement & Checkout Flow

1. Client POSTs request to '/api/users/addresses/{addressId}/orders/{paymentMethod}'.
2. OrderServiceImpl fetches the customer's cart. If empty, throws APIException.
3. Validates that the address exists and belongs to the customer.
4. Initializes an Order object, links the customer details, shipping address, and payment method.
5. Iterates through CartItems: maps to OrderItem loggers and deducts quantities directly from the Product inventory.
6. Updates Payment table status. Clears the cart items from the customer's cart, saves the Order, and returns OrderDTO.

Part 5: Complete Code Registry & Annotated Lines

File Path: pom.xml

COMPONENT PROFILE

Stereotype: **Java Helper Component**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'pom.xml' workflow executing smoothly.

Source Code Listing:

```
 1: <?xml version="1.0" encoding="UTF-8"?>
 2: <project xmlns="http://maven.apache.org/POM/4.0.0"
 3:          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 4:          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/mav
    en-4.0.0.xsd">
 5:     <modelVersion>4.0.0</modelVersion>
 6:
 7:     <parent>
 8:         <groupId>org.springframework.boot</groupId>
 9:         <artifactId>spring-boot-starter-parent</artifactId>
10:         <version>3.5.7</version>
11:         <relativePath/>
12:     </parent>
13:
14:     <groupId>com.ecommerce</groupId>
15:     <artifactId>sb-ecom</artifactId>
16:     <version>0.0.1-SNAPSHOT</version>
17:     <name>sb-ecom</name>
18:     <description>Spring Boot E-commerce Project</description>
19:
20:     <properties>
21:         <java.version>17</java.version>
22:     </properties>
23:
24:     <dependencies>
25:
26:         <!-- Core Spring Boot -->
27:         <dependency>
28:             <groupId>org.springframework.boot</groupId>
29:             <artifactId>spring-boot-starter-web</artifactId>
30:         </dependency>
31:
32:         <dependency>
33:             <groupId>org.springframework.boot</groupId>
34:             <artifactId>spring-boot-starter-data-jpa</artifactId>
35:         </dependency>
36:
37:         <dependency>
38:             <groupId>org.springframework.boot</groupId>
39:             <artifactId>spring-boot-starter-security</artifactId>
40:         </dependency>
41:
42:         <dependency>
43:             <groupId>org.springframework.boot</groupId>
44:             <artifactId>spring-boot-starter-validation</artifactId>
45:         </dependency>
46:
47:         <dependency>
48:             <groupId>org.postgresql</groupId>
49:             <artifactId>postgresql</artifactId>
50:             <scope>runtime</scope>
51:         </dependency>
52:
53:         <!-- Database -->
54:         <!-- <dependency>
55:             <groupId>com.h2database</groupId>
56:             <artifactId>h2</artifactId>
57:             <scope>runtime</scope>
```

```
        </dependency> -->

        <!-- JWT -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.12.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.12.5</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.12.5</version>
            <scope>runtime</scope>
        </dependency>

        <!-- ModelMapper -->
        <dependency>
            <groupId>org.modelmapper</groupId>
            <artifactId>modelmapper</artifactId>
            <version>3.2.4</version>
        </dependency>

        <!-- Lombok -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <!-- Testing -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

File Path: src/main/java/com/ecommerce/project/SbEcomApplication.java

COMPONENT PROFILE

Stereotype:	**Java Helper Component**
Active Annotations:	*@SpringBootApplication*

Technical Explanation of Code Logic:

This is the bootstrap class for the Spring Boot application. It contains the main method which launches the application context. It is annotated with @SpringBootApplication, which combines @Configuration, @EnableAutoConfiguration, and @ComponentScan. Additionally, it defines a ModelMapper @Bean to support DTO-to-entity mapping across controllers and services.

Source Code Listing:

```
 1: package com.ecommerce.project;
 2:
 3: import org.springframework.boot.SpringApplication;
 4: import org.springframework.boot.autoconfigure.SpringBootApplication;
 5:
 6: @SpringBootApplication
 7: public class SbEcomApplication {
 8:
 9:     public static void main(String[] args) {
10:         SpringApplication.run(SbEcomApplication.class, args);
11:     }
12:
13: }
```

File Path: src/main/java/com/ecommerce/project/config/AppConfig.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**
Active Annotations:	*@Bean, @Configuration*

Technical Explanation of Code Logic:

AppConfig defines core bean configurations for the application. Specifically, it declares the ModelMapper bean which simplifies class-to-class conversions (e.g., Product to ProductDTO) by matching field names automatically.

Source Code Listing:

```
 1: package com.ecommerce.project.config;
 2:
 3: import org.modelmapper.ModelMapper;
 4: import org.springframework.context.annotation.Bean;
 5: import org.springframework.context.annotation.Configuration;
 6:
 7: @Configuration
 8: public class AppConfig {
 9:
10:     @Bean
11:     public ModelMapper modelMapper(){
12:         return new ModelMapper();
13:     }
14: }
```

File Path: src/main/java/com/ecommerce/project/config/AppConstants.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**

Technical Explanation of Code Logic:

This utility file holds static final application constants (such as default page numbers, page sizes, sorting directions, and roles) used universally across the controllers and service implementations to avoid magic strings.

Source Code Listing:

```
1: package com.ecommerce.project.config;
2:
3: public class AppConstants {
4:     public static final String PAGE_NUMBER = "0";
5:     public static final String PAGE_SIZE = "50";
```

```
    public static final String SORT_CATEGORIES_BY = "categoryId";
    public static final String SORT_PRODUCTS_BY = "productId";
    public static final String SORT_DIR = "asc";
}
```

File Path: src/main/java/com/ecommerce/project/controller/AddressController.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**
Active Annotations:	*@DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping*

Technical Explanation of Code Logic:

AddressController manages user shipping addresses. Users can add, update, delete, and list their shipping addresses. These addresses are stored and associated with orders during checkout.

Source Code Listing:

```
package com.ecommerce.project.controller;

import com.ecommerce.project.model.User;
import com.ecommerce.project.payload.AddressDTO;
import com.ecommerce.project.service.AddressService;
import com.ecommerce.project.util.AuthUtil;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
public class AddressController {

    @Autowired
    AuthUtil authUtil;

    @Autowired
    AddressService addressService;

    @PostMapping("/addresses")
    public ResponseEntity<AddressDTO> createAddress(@Valid @RequestBody AddressDTO addressDTO){
        User user = authUtil.loggedInUser();
        AddressDTO savedAddressDTO = addressService.createAddress(addressDTO, user);
        return new ResponseEntity<>(savedAddressDTO, HttpStatus.CREATED);
    }

    @GetMapping("/addresses")
    public ResponseEntity<List<AddressDTO>> getAddresses(){
        List<AddressDTO> addressList = addressService.getAddresses();
        return new ResponseEntity<>(addressList, HttpStatus.OK);
    }

    @GetMapping("/addresses/{addressId}")
    public ResponseEntity<AddressDTO> getAddressById(@PathVariable Long addressId){
        AddressDTO addressDTO = addressService.getAddressesById(addressId);
        return new ResponseEntity<>(addressDTO, HttpStatus.OK);
    }


    @GetMapping("/users/addresses")
    public ResponseEntity<List<AddressDTO>> getUserAddresses(){
        User user = authUtil.loggedInUser();
        List<AddressDTO> addressList = addressService.getUserAddresses(user);
        return new ResponseEntity<>(addressList, HttpStatus.OK);
    }

    @PutMapping("/addresses/{addressId}")
```

```
53:     public ResponseEntity<AddressDTO> updateAddress(@PathVariable Long addressId
54:             , @RequestBody AddressDTO addressDTO){
55:         AddressDTO updatedAddress = addressService.updateAddress(addressId, addressDTO);
56:         return new ResponseEntity<>(updatedAddress, HttpStatus.OK);
57:     }
58:
59:     @DeleteMapping("/addresses/{addressId}")
60:     public ResponseEntity<String> updateAddress(@PathVariable Long addressId){
61:         String status = addressService.deleteAddress(addressId);
62:         return new ResponseEntity<>(status, HttpStatus.OK);
63:     }
64: }
```

File Path: src/main/java/com/ecommerce/project/controller/AuthController.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**
Active Annotations:	*@GetMapping, @PostMapping, @RequestBody, @RequestMapping, @RestController, @Valid*
Injected Dependencies:	jwtUtils (JwtUtils), authenticationManager (AuthenticationManager)

Technical Explanation of Code Logic:

AuthController handles guest/user authentication and identity management endpoints. It exposes POST '/api/auth/signup' for registering new accounts (validating unique usernames/emails and encoding passwords with BCrypt) and POST '/api/auth/signin' for user login. It authenticates credentials using Spring's AuthenticationManager, generates a JWT, embeds it inside an HttpOnly cookie, and returns the user's role and profile details.

Source Code Listing:

```
 1: package com.ecommerce.project.controller;
 2:
 3: import com.ecommerce.project.model.AppRole;
 4: import com.ecommerce.project.model.Role;
 5: import com.ecommerce.project.model.User;
 6: import com.ecommerce.project.repositories.RoleRepository;
 7: import com.ecommerce.project.repositories.UserRepository;
 8: import com.ecommerce.project.security.jwt.JwtUtils;
 9: import com.ecommerce.project.security.request.LoginRequest;
10: import com.ecommerce.project.security.request.SignupRequest;
11: import com.ecommerce.project.security.response.MessageResponse;
12: import com.ecommerce.project.security.response.UserInfoResponse;
13: import com.ecommerce.project.security.services.UserDetailsImpl;
14: import jakarta.validation.Valid;
15: import org.springframework.beans.factory.annotation.Autowired;
16: import org.springframework.http.HttpHeaders;
17: import org.springframework.http.HttpStatus;
18: import org.springframework.http.ResponseCookie;
19: import org.springframework.http.ResponseEntity;
20: import org.springframework.security.authentication.AuthenticationManager;
21: import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
22: import org.springframework.security.core.Authentication;
23: import org.springframework.security.core.AuthenticationException;
24: import org.springframework.security.core.context.SecurityContextHolder;
25: import org.springframework.security.crypto.password.PasswordEncoder;
26: import org.springframework.web.bind.annotation.*;
27:
28: import java.util.*;
29: import java.util.stream.Collectors;
30:
31: @RestController
32: @RequestMapping("/api/auth")
33: public class AuthController {
34:
35:     @Autowired
36:     private JwtUtils jwtUtils;
37:
38:     @Autowired
39:     private AuthenticationManager authenticationManager;
```

```
40:
41:     @Autowired
42:     UserRepository userRepository;
43:
44:     @Autowired
45:     RoleRepository roleRepository;
46:
47:     @Autowired
48:     PasswordEncoder encoder;
49:
50:     @PostMapping("/signin")
51:     public ResponseEntity<?> authenticateUser(@RequestBody LoginRequest loginRequest) {
52:         Authentication authentication;
53:         try {
54:             authentication = authenticationManager
55:                     .authenticate(new UsernamePasswordAuthenticationToken(loginRequest.getUsern
    ame(), loginRequest.getPassword()));
56:         } catch (AuthenticationException exception) {
57:             Map<String, Object> map = new HashMap<>();
58:             map.put("message", "Bad credentials");
59:             map.put("status", false);
60:             return new ResponseEntity<Object>(map, HttpStatus.NOT_FOUND);
61:         }
62:
63:         SecurityContextHolder.getContext().setAuthentication(authentication);
64:
65:         UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
66:
67:         ResponseCookie jwtCookie = jwtUtils.generateJwtCookie(userDetails);
68:
69:         List<String> roles = userDetails.getAuthorities().stream()
70:                 .map(item -> item.getAuthority())
71:                 .collect(Collectors.toList());
72:
73:         UserInfoResponse response = new UserInfoResponse(userDetails.getId(),
74:                 userDetails.getUsername(), roles, jwtCookie.toString());
75:
76:         return ResponseEntity.ok().header(HttpHeaders.SET_COOKIE,
77:                 jwtCookie.toString())
78:                 .body(response);
79:     }
80:
81:     @PostMapping("/signup")
82:     public ResponseEntity<?> registerUser(@Valid @RequestBody SignupRequest signUpRequest) {
83:         if (userRepository.existsByUserName(signUpRequest.getUsername())) {
84:             return ResponseEntity.badRequest().body(new MessageResponse("Error: Username is alr
    eady taken!"));
85:         }
86:
87:         if (userRepository.existsByEmail(signUpRequest.getEmail())) {
88:             return ResponseEntity.badRequest().body(new MessageResponse("Error: Email is alread
    y in use!"));
89:         }
90:
91:         // Create new user's account
92:         User user = new User(signUpRequest.getUsername(),
93:                 signUpRequest.getEmail(),
94:                 encoder.encode(signUpRequest.getPassword()));
95:
96:         Set<String> strRoles = signUpRequest.getRole();
97:         Set<Role> roles = new HashSet<>();
98:
99:         if (strRoles == null) {
100:             Role userRole = roleRepository.findByRoleName(AppRole.ROLE_USER)
101:                     .orElseThrow(() -> new RuntimeException("Error: Role is not found."));
102:             roles.add(userRole);
103:         } else {
104:             strRoles.forEach(role -> {
105:                 switch (role) {
106:                     case "admin":
107:                         Role adminRole = roleRepository.findByRoleName(AppRole.ROLE_ADMIN)
108:                                 .orElseThrow(() -> new RuntimeException("Error: Role is not fou
    nd."));
109:                         roles.add(adminRole);
110:
111:                         break;
112:                     case "seller":
113:                         Role modRole = roleRepository.findByRoleName(AppRole.ROLE_SELLER)
114:                                 .orElseThrow(() -> new RuntimeException("Error: Role is not fou
    nd."));
```

```
115:                         roles.add(modRole);
116:
117:                         break;
118:                     default:
119:                         Role userRole = roleRepository.findByRoleName(AppRole.ROLE_USER)
120:                                 .orElseThrow(() -> new RuntimeException("Error: Role is not fou
    nd."));
121:                         roles.add(userRole);
122:                 }
123:             });
124:         }
125:
126:         user.setRoles(roles);
127:         userRepository.save(user);
128:
129:         return ResponseEntity.ok(new MessageResponse("User registered successfully!"));
130:     }
131:
132:     @GetMapping("/username")
133:     public String currentUserName(Authentication authentication){
134:         if (authentication != null)
135:             return authentication.getName();
136:         else
137:             return "";
138:     }
139:
140:
141:     @GetMapping("/user")
142:     public ResponseEntity<?> getUserDetails(Authentication authentication){
143:         UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
144:
145:         List<String> roles = userDetails.getAuthorities().stream()
146:                 .map(item -> item.getAuthority())
147:                 .collect(Collectors.toList());
148:
149:         UserInfoResponse response = new UserInfoResponse(userDetails.getId(),
150:                 userDetails.getUsername(), roles);
151:
152:         return ResponseEntity.ok().body(response);
153:     }
154:
155:     @PostMapping("/signout")
156:     public ResponseEntity<?> signoutUser(){
157:         ResponseCookie cookie = jwtUtils.getCleanJwtCookie();
158:         return ResponseEntity.ok().header(HttpHeaders.SET_COOKIE,
159:                         cookie.toString())
160:                 .body(new MessageResponse("You've been signed out!"));
161:     }
162: }
```

File Path: src/main/java/com/ecommerce/project/controller/CartController.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**
Active Annotations:	*@DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestMapping, @RestControlle*
Injected Dependencies:	cartRepository (CartRepository), authUtil (AuthUtil), cartService (CartService)

Technical Explanation of Code Logic:

CartController defines endpoints for managing shopping carts. It exposes POST endpoints to add a product to a cart, GET endpoints to fetch the logged-in user's cart or retrieve all carts, PUT endpoints to update product quantities in a cart, and DELETE endpoints to remove items from a cart. It uses Spring Security to identify the active user context.

Source Code Listing:

```
1: package com.ecommerce.project.controller;
2:
3: import com.ecommerce.project.model.Cart;
4: import com.ecommerce.project.payload.CartDTO;
```

```
 5: import com.ecommerce.project.repositories.CartRepository;
 6: import com.ecommerce.project.service.CartService;
 7: import com.ecommerce.project.util.AuthUtil;
 8: import org.springframework.beans.factory.annotation.Autowired;
 9: import org.springframework.http.HttpStatus;
10: import org.springframework.http.ResponseEntity;
11: import org.springframework.web.bind.annotation.*;
12:
13: import java.util.List;
14:
15: @RestController
16: @RequestMapping("/api")
17: public class CartController {
18:
19:     @Autowired
20:     private CartRepository cartRepository;
21:
22:     @Autowired
23:     private AuthUtil authUtil;
24:
25:     @Autowired
26:     private CartService cartService;
27:
28:     @PostMapping("/carts/products/{productId}/quantity/{quantity}")
29:     public ResponseEntity<CartDTO> addProductToCart(@PathVariable Long productId,
30:                                                     @PathVariable Integer quantity){
31:         CartDTO cartDTO = cartService.addProductToCart(productId, quantity);
32:         return new ResponseEntity<CartDTO>(cartDTO, HttpStatus.CREATED);
33:     }
34:
35:     @GetMapping("/carts")
36:     public ResponseEntity<List<CartDTO>> getCarts() {
37:         List<CartDTO> cartDTOs = cartService.getAllCarts();
38:         return new ResponseEntity<List<CartDTO>>(cartDTOs, HttpStatus.FOUND);
39:     }
40:
41:     @GetMapping("/carts/users/cart")
42:     public ResponseEntity<CartDTO> getCartById(){
43:         String emailId = authUtil.loggedInEmail();
44:         Cart cart = cartRepository.findCartByEmail(emailId);
45:         Long cartId = cart.getCartId();
46:         CartDTO cartDTO = cartService.getCart(emailId, cartId);
47:         return new ResponseEntity<CartDTO>(cartDTO, HttpStatus.OK);
48:     }
49:
50:     @PutMapping("/cart/products/{productId}/quantity/{operation}")
51:     public ResponseEntity<CartDTO> updateCartProduct(@PathVariable Long productId,
52:                                                      @PathVariable String operation) {
53:
54:         CartDTO cartDTO = cartService.updateProductQuantityInCart(productId,
55:                 operation.equalsIgnoreCase("delete") ? -1 : 1);
56:
57:         return new ResponseEntity<CartDTO>(cartDTO, HttpStatus.OK);
58:     }
59:
60:     @DeleteMapping("/carts/{cartId}/product/{productId}")
61:     public ResponseEntity<String> deleteProductFromCart(@PathVariable Long cartId,
62:                                                         @PathVariable Long productId) {
63:         String status = cartService.deleteProductFromCart(cartId, productId);
64:
65:         return new ResponseEntity<String>(status, HttpStatus.OK);
66:     }
67: }
```

File Path: src/main/java/com/ecommerce/project/controller/CategoryController.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**
Active Annotations:	*@DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping*
Injected Dependencies:	categoryService (CategoryService)

Technical Explanation of Code Logic:

CategoryController provides endpoints for managing category entities. It enables administrative creation (POST), updating (PUT), and deletion (DELETE) of categories, and public access (GET) to list categories. It supports pagination, sorting, and search filtering through request parameters.

Source Code Listing:

```
 1: package com.ecommerce.project.controller;
 2:
 3: import com.ecommerce.project.config.AppConstants;
 4: import com.ecommerce.project.payload.CategoryDTO;
 5: import com.ecommerce.project.payload.CategoryResponse;
 6: import com.ecommerce.project.service.CategoryService;
 7: import jakarta.validation.Valid;
 8: import org.springframework.beans.factory.annotation.Autowired;
 9: import org.springframework.http.HttpStatus;
10: import org.springframework.http.ResponseEntity;
11: import org.springframework.web.bind.annotation.*;
12:
13: @RestController
14: @RequestMapping("/api")
15: public class CategoryController {
16:
17:     @Autowired
18:     private CategoryService categoryService;
19:
20:     @GetMapping("/public/categories")
21:     public ResponseEntity<CategoryResponse> getAllCategories(
22:             @RequestParam(name = "pageNumber", defaultValue = AppConstants.PAGE_NUMBER, require
    d = false) Integer pageNumber,
23:             @RequestParam(name = "pageSize", defaultValue = AppConstants.PAGE_SIZE, required =
    false) Integer pageSize,
24:             @RequestParam(name = "sortBy", defaultValue = AppConstants.SORT_CATEGORIES_BY, requ
    ired = false) String sortBy,
25:             @RequestParam(name = "sortOrder", defaultValue = AppConstants.SORT_DIR, required =
    false) String sortOrder) {
26:         CategoryResponse categoryResponse = categoryService.getAllCategories(pageNumber, pageSi
    ze, sortBy, sortOrder);
27:         return new ResponseEntity<>(categoryResponse, HttpStatus.OK);
28:     }
29:
30:     @PostMapping("/public/categories")
31:     public ResponseEntity<CategoryDTO> createCategory(@Valid @RequestBody CategoryDTO categoryD
    TO){
32:         CategoryDTO savedCategoryDTO = categoryService.createCategory(categoryDTO);
33:         return new ResponseEntity<>(savedCategoryDTO, HttpStatus.CREATED);
34:     }
35:
36:     @DeleteMapping("/admin/categories/{categoryId}")
37:     public ResponseEntity<CategoryDTO> deleteCategory(@PathVariable Long categoryId){
38:             CategoryDTO deletedCategory = categoryService.deleteCategory(categoryId);
39:             return new ResponseEntity<>(deletedCategory, HttpStatus.OK);
40:     }
41:
42:
43:     @PutMapping("/public/categories/{categoryId}")
44:     public ResponseEntity<CategoryDTO> updateCategory(@Valid @RequestBody CategoryDTO categoryD
    TO,
45:                                                       @PathVariable Long categoryId){
46:             CategoryDTO savedCategoryDTO = categoryService.updateCategory(categoryDTO, category
    Id);
47:             return new ResponseEntity<>(savedCategoryDTO, HttpStatus.OK);
48:     }
49: }
```

File Path: src/main/java/com/ecommerce/project/controller/OrderController.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**

Active Annotations:	*@PathVariable, @PostMapping, @RequestBody, @RequestMapping, @RestController*
Injected Dependencies:	orderService (OrderService), authUtil (AuthUtil)

Technical Explanation of Code Logic:

OrderController handles checkout operations. It exposes POST '/api/users/addresses/{addressId}/orders/{paymentMethod}' to place an order, mapping the user's cart items to a persistent Order. It also exposes GET endpoints to search order histories by user or category.

Source Code Listing:

```
 1: package com.ecommerce.project.controller;
 2:
 3: import com.ecommerce.project.payload.OrderDTO;
 4: import com.ecommerce.project.payload.OrderRequestDTO;
 5: import com.ecommerce.project.service.OrderService;
 6: import com.ecommerce.project.util.AuthUtil;
 7: import org.springframework.beans.factory.annotation.Autowired;
 8: import org.springframework.http.HttpStatus;
 9: import org.springframework.http.ResponseEntity;
10: import org.springframework.web.bind.annotation.*;
11:
12: @RestController
13: @RequestMapping("/api")
14: public class OrderController {
15:
16:     @Autowired
17:     private OrderService orderService;
18:
19:     @Autowired
20:     private AuthUtil authUtil;
21:
22:     @PostMapping("/order/users/payments/{paymentMethod}")
23:     public ResponseEntity<OrderDTO> orderProducts(@PathVariable String paymentMethod, @RequestB
    ody OrderRequestDTO orderRequestDTO) {
24:         String emailId = authUtil.loggedInEmail();
25:         OrderDTO order = orderService.placeOrder(
26:                 emailId,
27:                 orderRequestDTO.getAddressId(),
28:                 paymentMethod,
29:                 orderRequestDTO.getPgName(),
30:                 orderRequestDTO.getPgPaymentId(),
31:                 orderRequestDTO.getPgStatus(),
32:                 orderRequestDTO.getPgResponseMessage()
33:         );
34:         return new ResponseEntity<>(order, HttpStatus.CREATED);
35:     }
36: }
```

File Path: src/main/java/com/ecommerce/project/controller/ProductController.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**
Active Annotations:	*@DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping*

Technical Explanation of Code Logic:

ProductController exposes endpoints for product management. It supports admin/seller actions for adding, updating, and deleting products, and public actions for browsing products. It features complex pagination, custom sorting, category-based filtering, and endpoints for uploading/serving product images.

Source Code Listing:

```
1: package com.ecommerce.project.controller;
2:
3: import com.ecommerce.project.config.AppConstants;
```

```
 4: import com.ecommerce.project.payload.ProductDTO;
 5: import com.ecommerce.project.payload.ProductResponse;
 6: import com.ecommerce.project.service.ProductService;
 7: import jakarta.validation.Valid;
 8: import org.springframework.beans.factory.annotation.Autowired;
 9: import org.springframework.http.HttpStatus;
10: import org.springframework.http.ResponseEntity;
11: import org.springframework.web.bind.annotation.*;
12: import org.springframework.web.multipart.MultipartFile;
13:
14: import java.io.IOException;
15:
16: @RestController
17: @RequestMapping("/api")
18: public class ProductController {
19:
20:     @Autowired
21:     ProductService productService;
22:
23:     @PostMapping("/admin/categories/{categoryId}/product")
24:     public ResponseEntity<ProductDTO> addProduct(@Valid @RequestBody ProductDTO productDTO,
25:                                                  @PathVariable Long categoryId){
26:         ProductDTO savedProductDTO = productService.addProduct(categoryId, productDTO);
27:         return new ResponseEntity<>(savedProductDTO, HttpStatus.CREATED);
28:     }
29:
30:     @GetMapping("/public/products")
31:     public ResponseEntity<ProductResponse> getAllProducts(
32:             @RequestParam(name = "pageNumber", defaultValue = AppConstants.PAGE_NUMBER, require
    d = false) Integer pageNumber,
33:             @RequestParam(name = "pageSize", defaultValue = AppConstants.PAGE_SIZE, required =
    false) Integer pageSize,
34:             @RequestParam(name = "sortBy", defaultValue = AppConstants.SORT_PRODUCTS_BY, requir
    ed = false) String sortBy,
35:             @RequestParam(name = "sortOrder", defaultValue = AppConstants.SORT_DIR, required =
    false) String sortOrder
36:     ){
37:         ProductResponse productResponse = productService.getAllProducts(pageNumber, pageSize, s
    ortBy, sortOrder);
38:         return new ResponseEntity<>(productResponse,HttpStatus.OK);
39:     }
40:
41:     @GetMapping("/public/categories/{categoryId}/products")
42:     public ResponseEntity<ProductResponse> getProductsByCategory(@PathVariable Long categoryId,
43:                                                                   @RequestParam(name = "pageNumb
    er", defaultValue = AppConstants.PAGE_NUMBER, required = false) Integer pageNumber,
44:                                                                   @RequestParam(name = "pageSize
    ", defaultValue = AppConstants.PAGE_SIZE, required = false) Integer pageSize,
45:                                                                   @RequestParam(name = "sortBy",
     defaultValue = AppConstants.SORT_PRODUCTS_BY, required = false) String sortBy,
46:                                                                   @RequestParam(name = "sortOrde
    r", defaultValue = AppConstants.SORT_DIR, required = false) String sortOrder){
47:         ProductResponse productResponse = productService.searchByCategory(categoryId, pageNumbe
    r, pageSize, sortBy, sortOrder);
48:         return new ResponseEntity<>(productResponse, HttpStatus.OK);
49:     }
50:
51:     @GetMapping("/public/products/keyword/{keyword}")
52:     public ResponseEntity<ProductResponse> getProductsByKeyword(@PathVariable String keyword,
53:                                                                  @RequestParam(name = "pageNumbe
    r", defaultValue = AppConstants.PAGE_NUMBER, required = false) Integer pageNumber,
54:                                                                  @RequestParam(name = "pageSize"
    , defaultValue = AppConstants.PAGE_SIZE, required = false) Integer pageSize,
55:                                                                  @RequestParam(name = "sortBy",
    defaultValue = AppConstants.SORT_PRODUCTS_BY, required = false) String sortBy,
56:                                                                  @RequestParam(name = "sortOrder
    ", defaultValue = AppConstants.SORT_DIR, required = false) String sortOrder){
57:         ProductResponse productResponse = productService.searchProductByKeyword(keyword, pageNu
    mber, pageSize, sortBy, sortOrder);
58:         return new ResponseEntity<>(productResponse, HttpStatus.FOUND);
59:     }
60:
61:     @PutMapping("/admin/products/{productId}")
62:     public ResponseEntity<ProductDTO> updateProduct(@Valid @RequestBody ProductDTO productDTO,
63:                                                     @PathVariable Long productId){
64:         ProductDTO updatedProductDTO = productService.updateProduct(productId, productDTO);
65:         return new ResponseEntity<>(updatedProductDTO, HttpStatus.OK);
66:     }
67:
68:     @DeleteMapping("/admin/products/{productId}")
```

```
69:     public ResponseEntity<ProductDTO> deleteProduct(@PathVariable Long productId){
70:         ProductDTO deletedProduct = productService.deleteProduct(productId);
71:         return new ResponseEntity<>(deletedProduct, HttpStatus.OK);
72:     }
73:
74:     @PutMapping("/products/{productId}/image")
75:     public ResponseEntity<ProductDTO> updateProductImage(@PathVariable Long productId,
76:                                                          @RequestParam("image")MultipartFile im
    age) throws IOException {
77:         ProductDTO updatedProduct = productService.updateProductImage(productId, image);
78:         return new ResponseEntity<>(updatedProduct, HttpStatus.OK);
79:     }
80: }
```

File Path: src/main/java/com/ecommerce/project/exceptions/APIException.java

COMPONENT PROFILE

Stereotype:	**Exception Handler / Custom Exception**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'APIException' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.exceptions;
 2:
 3: public class APIException extends RuntimeException {
 4:     private static final long serialVersionUID = 1L;
 5:
 6:     public APIException() {
 7:     }
 8:
 9:     public APIException(String message) {
10:         super(message);
11:     }
12: }
```

File Path: src/main/java/com/ecommerce/project/exceptions/MyGlobalExceptionHandler.java

COMPONENT PROFILE

Stereotype:	**REST Controller (API Gateway / Endpoint Handler)**
Active Annotations:	*@ExceptionHandler, @RestControllerAdvice*

Technical Explanation of Code Logic:

Exposes endpoints for handling client queries and requests related to 'MyGlobalExceptionHandler'. It validates requests, coordinates with service implementations, and formats API JSON structures.

Source Code Listing:

```
 1: package com.ecommerce.project.exceptions;
 2:
 3:
 4: import com.ecommerce.project.payload.APIResponse;
 5: import org.springframework.http.HttpStatus;
 6: import org.springframework.http.ResponseEntity;
 7: import org.springframework.validation.FieldError;
 8: import org.springframework.web.bind.MethodArgumentNotValidException;
 9: import org.springframework.web.bind.annotation.ExceptionHandler;
10: import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
11:
12: import java.util.HashMap;
13: import java.util.Map;
14:
15: @RestControllerAdvice
16: public class MyGlobalExceptionHandler {
17:
18:     @ExceptionHandler(MethodArgumentNotValidException.class)
19:     public ResponseEntity<Map<String, String>> myMethodArgumentNotValidException(MethodArgument
    NotValidException e) {
20:         Map<String, String> response = new HashMap<>();
21:         e.getBindingResult().getAllErrors().forEach(err -> {
22:             String fieldName = ((FieldError) err).getField();
23:             String message = err.getDefaultMessage();
24:             response.put(fieldName, message);
25:         });
26:         return new ResponseEntity<Map<String, String>>(response,
27:                 HttpStatus.BAD_REQUEST);
28:     }
29:
30:     @ExceptionHandler(ResourceNotFoundException.class)
31:     public ResponseEntity<APIResponse> myResourceNotFoundException(ResourceNotFoundException e)
     {
32:         String message = e.getMessage();
33:         APIResponse apiResponse = new APIResponse(message, false);
34:         return new ResponseEntity<>(apiResponse, HttpStatus.NOT_FOUND);
35:     }
36:
37:     @ExceptionHandler(APIException.class)
38:     public ResponseEntity<APIResponse> myAPIException(APIException e) {
39:         String message = e.getMessage();
40:         APIResponse apiResponse = new APIResponse(message, false);
41:         return new ResponseEntity<>(apiResponse, HttpStatus.BAD_REQUEST);
42:     }
43: }
```

File Path: src/main/java/com/ecommerce/project/exceptions/ResourceNotFoundException.java

COMPONENT PROFILE

Stereotype: **Exception Handler / Custom Exception**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'ResourceNotFoundException' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.exceptions;
 2:
 3: public class ResourceNotFoundException extends RuntimeException {
 4:     String resourceName;
 5:     String field;
 6:     String fieldName;
 7:     Long fieldId;
 8:
 9:     public ResourceNotFoundException() {
10:     }
11:
12:     public ResourceNotFoundException(String resourceName, String field, String fieldName) {
13:         super(String.format("%s not found with %s: %s", resourceName, field, fieldName));
14:         this.resourceName = resourceName;
15:         this.field = field;
16:         this.fieldName = fieldName;
17:     }
18:
19:     public ResourceNotFoundException(String resourceName, String field, Long fieldId) {
20:         super(String.format("%s not found with %s: %d", resourceName, field, fieldId));
21:         this.resourceName = resourceName;
22:         this.field = field;
23:         this.fieldId = fieldId;
```

```
24:     }
25: }
```

File Path: src/main/java/com/ecommerce/project/model/Address.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @JoinColumn, @ManyToOne, @NotBlank, @Size, @Table*
Injected Dependencies:	user (User)

Technical Explanation of Code Logic:

This JPA Entity class 'Address' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import jakarta.validation.constraints.NotBlank;
 5: import jakarta.validation.constraints.Size;
 6: import lombok.AllArgsConstructor;
 7: import lombok.Data;
 8: import lombok.NoArgsConstructor;
 9: import lombok.ToString;
10:
11: import java.util.ArrayList;
12: import java.util.List;
13:
14: @Entity
15: @Table(name = "addresses")
16: @Data
17: @NoArgsConstructor
18: @AllArgsConstructor
19: public class Address {
20:     @Id
21:     @GeneratedValue(strategy = GenerationType.IDENTITY)
22:     private Long addressId;
23:
24:     @NotBlank
25:     @Size(min = 5, message = "Street name must be atleast 5 characters")
26:     private String street;
27:
28:     @NotBlank
29:     @Size(min = 5, message = "Building name must be atleast 5 characters")
30:     private String buildingName;
31:
32:     @NotBlank
33:     @Size(min = 4, message = "City name must be atleast 4 characters")
34:     private String city;
35:
36:     @NotBlank
37:     @Size(min = 2, message = "State name must be atleast 2 characters")
38:     private String state;
39:
40:     @NotBlank
41:     @Size(min = 2, message = "Country name must be atleast 2 characters")
42:     private String country;
43:
44:     @NotBlank
45:     @Size(min = 5, message = "Pincode must be atleast 5 characters")
46:     private String pincode;
47:
48:     @ManyToOne
49:     @JoinColumn(name = "user_id")
50:     private User user;
51:
52:     public Address(String street, String buildingName, String city, String state, String countr
```

```
    y, String pincode) {
53:         this.street = street;
54:         this.buildingName = buildingName;
55:         this.city = city;
56:         this.state = state;
57:         this.country = country;
58:         this.pincode = pincode;
59:     }
60: }
```

File Path: src/main/java/com/ecommerce/project/model/AppRole.java

COMPONENT PROFILE

Stereotype:	**Enumeration Type**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'AppRole' workflow executing smoothly.

Source Code Listing:

```
1: package com.ecommerce.project.model;
2:
3: public enum AppRole {
4:     ROLE_USER,
5:     ROLE_SELLER,
6:     ROLE_ADMIN
7: }
```

File Path: src/main/java/com/ecommerce/project/model/Cart.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @JoinColumn, @OneToMany, @OneToOne, @Table*
Injected Dependencies:	user (User)

Technical Explanation of Code Logic:

This JPA Entity class 'Cart' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
1: package com.ecommerce.project.model;
2:
3: import jakarta.persistence.*;
4: import lombok.AllArgsConstructor;
5: import lombok.Data;
6: import lombok.NoArgsConstructor;
7:
8: import java.util.ArrayList;
9: import java.util.List;
10:
11: @Entity
12: @Data
13: @Table(name = "carts")
14: @NoArgsConstructor
15: @AllArgsConstructor
16: public class Cart {
17:     @Id
18:     @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long cartId;

    @OneToOne
    @JoinColumn(name = "user_id")
    private User user;

    @OneToMany(mappedBy = "cart", cascade = {CascadeType.PERSIST, CascadeType.MERGE, CascadeTyp
e.REMOVE}, orphanRemoval = true)
    private List<CartItem> cartItems = new ArrayList<>();

    private Double totalPrice = 0.0;
}
```

File Path: src/main/java/com/ecommerce/project/model/CartItem.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @JoinColumn, @ManyToOne, @Table*
Injected Dependencies:	cart (Cart), product (Product)

Technical Explanation of Code Logic:

This JPA Entity class 'CartItem' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
package com.ecommerce.project.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@Table(name = "cart_items")
@NoArgsConstructor
@AllArgsConstructor
public class CartItem {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long cartItemId;

    @ManyToOne
    @JoinColumn(name = "cart_id")
    private Cart cart;

    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;

    private Integer quantity;
    private double discount;
    private double productPrice;
}
```

File Path: src/main/java/com/ecommerce/project/model/Category.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**

Active Annotations:	*@Data, @Entity, @NotBlank, @OneToMany, @Size*

Technical Explanation of Code Logic:

This JPA Entity class 'Category' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import jakarta.validation.constraints.NotBlank;
 5: import jakarta.validation.constraints.Size;
 6: import lombok.AllArgsConstructor;
 7: import lombok.Data;
 8: import lombok.NoArgsConstructor;
 9:
10: import java.util.List;
11:
12: @Entity(name = "categories")
13: @Data
14: @NoArgsConstructor
15: @AllArgsConstructor
16: public class Category {
17:     @Id
18:     @GeneratedValue(strategy = GenerationType.IDENTITY)
19:     private Long categoryId;
20:
21:     @NotBlank
22:     @Size(min = 5, message = "Category name must contain atleast 5 characters")
23:     private String categoryName;
24:
25:     @OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
26:     private List<Product> products;
27: }
```

File Path: src/main/java/com/ecommerce/project/model/Order.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Email, @Entity, @JoinColumn, @ManyToOne, @OneToMany, @OneToOne, @Table*
Injected Dependencies:	orderDate (LocalDate), payment (Payment), address (Address)

Technical Explanation of Code Logic:

This JPA Entity class 'Order' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import jakarta.validation.constraints.Email;
 5: import lombok.AllArgsConstructor;
 6: import lombok.Data;
 7: import lombok.NoArgsConstructor;
 8:
 9: import java.time.LocalDate;
10: import java.util.ArrayList;
11: import java.util.List;
12:
13: @Entity
14: @Table(name = "orders")
15: @Data
16: @NoArgsConstructor
17: @AllArgsConstructor
```

```
18: public class Order {
19:
20:     @Id
21:     @GeneratedValue(strategy = GenerationType.IDENTITY)
22:     private Long orderId;
23:
24:     @Email
25:     @Column(nullable = false)
26:     private String email;
27:
28:     @OneToMany(mappedBy = "order", cascade = { CascadeType.PERSIST, CascadeType.MERGE })
29:     private List<OrderItem> orderItems = new ArrayList<>();
30:
31:     private LocalDate orderDate;
32:
33:     @OneToOne
34:     @JoinColumn(name = "payment_id")
35:     private Payment payment;
36:
37:     private Double totalAmount;
38:     private String orderStatus;
39:
40:     // Reference to Address
41:     @ManyToOne
42:     @JoinColumn(name = "address_id")
43:     private Address address;
44: }
```

File Path: src/main/java/com/ecommerce/project/model/OrderItem.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @JoinColumn, @ManyToOne, @Table*
Injected Dependencies:	product (Product), order (Order)

Technical Explanation of Code Logic:

This JPA Entity class 'OrderItem' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import lombok.AllArgsConstructor;
 5: import lombok.Data;
 6: import lombok.NoArgsConstructor;
 7:
 8: @Entity
 9: @Data
10: @Table(name = "order_items")
11: @AllArgsConstructor
12: @NoArgsConstructor
13: public class OrderItem {
14:
15:     @Id
16:     @GeneratedValue(strategy = GenerationType.IDENTITY)
17:     private Long orderItemId;
18:
19:     @ManyToOne
20:     @JoinColumn(name = "product_id")
21:     private Product product;
22:
23:     @ManyToOne
24:     @JoinColumn(name = "order_id")
25:     private Order order;
26:
27:     private Integer quantity;
```

```
28:     private double discount;
29:     private double orderedProductPrice;
30:
31: }
```

File Path: src/main/java/com/ecommerce/project/model/Payment.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @NotBlank, @OneToOne, @Size, @Table*
Injected Dependencies:	order (Order)

Technical Explanation of Code Logic:

This JPA Entity class 'Payment' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import jakarta.validation.constraints.NotBlank;
 5: import jakarta.validation.constraints.Size;
 6: import lombok.AllArgsConstructor;
 7: import lombok.Data;
 8: import lombok.NoArgsConstructor;
 9:
10: @Entity
11: @Table(name = "payments")
12: @Data
13: @NoArgsConstructor
14: @AllArgsConstructor
15: public class Payment {
16:
17:     @Id
18:     @GeneratedValue(strategy = GenerationType.IDENTITY)
19:     private Long paymentId;
20:
21:     @OneToOne(mappedBy = "payment", cascade = { CascadeType.PERSIST, CascadeType.MERGE })
22:     private Order order;
23:
24:     @NotBlank
25:     @Size(min = 4, message = "Payment method must contain at least 4 characters")
26:     private String paymentMethod;
27:
28:     private String pgPaymentId;
29:     private String pgStatus;
30:     private String pgResponseMessage;
31:
32:     private String pgName;
33:
34:
35:     public Payment(String paymentMethod, String pgPaymentId, String pgStatus,
36:                    String pgResponseMessage, String pgName) {
37:         this.paymentMethod = paymentMethod;
38:         this.pgPaymentId = pgPaymentId;
39:         this.pgStatus = pgStatus;
40:         this.pgResponseMessage = pgResponseMessage;
41:         this.pgName = pgName;
42:     }
43: }
```

File Path: src/main/java/com/ecommerce/project/model/Product.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @JoinColumn, @ManyToOne, @NotBlank, @OneToMany, @Size, @Table, @ToString*
Injected Dependencies:	category (Category), user (User)

Technical Explanation of Code Logic:

This JPA Entity class 'Product' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import jakarta.validation.constraints.NotBlank;
 5: import jakarta.validation.constraints.Size;
 6: import lombok.AllArgsConstructor;
 7: import lombok.Data;
 8: import lombok.NoArgsConstructor;
 9: import lombok.ToString;
10:
11: import java.util.ArrayList;
12: import java.util.List;
13:
14: @Entity
15: @Data
16: @NoArgsConstructor
17: @AllArgsConstructor
18: @Table(name = "products")
19: @ToString
20: public class Product {
21:
22:     @Id
23:     @GeneratedValue(strategy = GenerationType.AUTO)
24:     private Long productId;
25:
26:     @NotBlank
27:     @Size(min = 3, message = "Product name must contain atleast 3 characters")
28:     private String productName;
29:     private String image;
30:
31:     @NotBlank
32:     @Size(min = 6, message = "Product description must contain atleast 6 characters")
33:     private String description;
34:     private Integer quantity;
35:     private double price;
36:     private double discount;
37:     private double specialPrice;
38:
39:     @ManyToOne
40:     @JoinColumn(name = "category_id")
41:     private Category category;
42:
43:     @ManyToOne
44:     @JoinColumn(name = "seller_id")
45:     private User user;
46:
47:     @OneToMany(mappedBy = "product", cascade = {CascadeType.PERSIST, CascadeType.MERGE}, fetch
    = FetchType.EAGER)
48:     private List<CartItem> products = new ArrayList<>();
49: }
```

File Path: src/main/java/com/ecommerce/project/model/Role.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Data, @Entity, @Enumerated, @Table, @ToString*
Injected Dependencies:	roleName (AppRole)

Technical Explanation of Code Logic:

This JPA Entity class 'Role' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import lombok.AllArgsConstructor;
 5: import lombok.Data;
 6: import lombok.NoArgsConstructor;
 7: import lombok.ToString;
 8:
 9: @Entity
10: @NoArgsConstructor
11: @AllArgsConstructor
12: @Data
13: @Table(name = "roles")
14: public class Role {
15:
16:     @Id
17:     @GeneratedValue(strategy = GenerationType.IDENTITY)
18:     @Column(name = "role_id")
19:     private Integer roleId;
20:
21:     @ToString.Exclude
22:     @Enumerated(EnumType.STRING)
23:     @Column(length = 20, name = "role_name")
24:     private AppRole roleName;
25:
26:     public Role(AppRole roleName) {
27:         this.roleName = roleName;
28:     }
29: }
```

File Path: src/main/java/com/ecommerce/project/model/User.java

COMPONENT PROFILE

Stereotype:	**JPA Entity (Database Model Mapping)**
Active Annotations:	*@Email, @Entity, @JoinColumn, @JoinTable, @ManyToMany, @NotBlank, @OneToMany, @OneToOne, @Size, @Ta*
Injected Dependencies:	cart (Cart)

Technical Explanation of Code Logic:

This JPA Entity class 'User' maps code objects to the PostgreSQL schema. It represents database structure and configures constraints, primary keys, fields, and foreign relationships to support data storage.

Source Code Listing:

```
 1: package com.ecommerce.project.model;
 2:
 3: import jakarta.persistence.*;
 4: import jakarta.validation.constraints.Email;
 5: import jakarta.validation.constraints.NotBlank;
 6: import jakarta.validation.constraints.Size;
 7: import lombok.*;
 8:
 9: import java.util.ArrayList;
10: import java.util.HashSet;
11: import java.util.List;
```

```
12: import java.util.Set;
13:
14: @Entity
15: @NoArgsConstructor
16: @Table(name = "users",
17:         uniqueConstraints = {
18:         @UniqueConstraint(columnNames = "username"),
19:         @UniqueConstraint(columnNames = "email")
20:         })
21: @Getter
22: @Setter
23: public class User {
24:     @Id
25:     @GeneratedValue(strategy = GenerationType.IDENTITY)
26:     @Column(name = "user_id")
27:     private Long userId;
28:
29:     @NotBlank
30:     @Size(max = 20)
31:     @Column(name = "username")
32:     private String userName;
33:
34:     @NotBlank
35:     @Size(max = 50)
36:     @Email
37:     @Column(name = "email")
38:     private String email;
39:
40:     @NotBlank
41:     @Size(max = 120)
42:     @Column(name = "password")
43:     private String password;
44:
45:     public User(String userName, String email, String password) {
46:         this.userName = userName;
47:         this.email = email;
48:         this.password = password;
49:     }
50:
51:     @Setter
52:     @Getter
53:     @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE},
54:                 fetch = FetchType.EAGER)
55:     @JoinTable(name = "user_role",
56:                 joinColumns = @JoinColumn(name = "user_id"),
57:                 inverseJoinColumns = @JoinColumn(name = "role_id"))
58:     private Set<Role> roles = new HashSet<>();
59:
60:     @Getter
61:     @Setter
62:     @OneToMany(mappedBy = "user", cascade = {CascadeType.PERSIST, CascadeType.MERGE}, orphanRem
    oval = true)
63: //    @JoinTable(name = "user_address",
64: //                joinColumns = @JoinColumn(name = "user_id"),
65: //                inverseJoinColumns = @JoinColumn(name = "address_id"))
66:     private List<Address> addresses = new ArrayList<>();
67:
68:     @ToString.Exclude
69:     @OneToOne(mappedBy = "user", cascade = { CascadeType.PERSIST, CascadeType.MERGE}, orphanRem
    oval = true)
70:     private Cart cart;
71:
72:     @ToString.Exclude
73:     @OneToMany(mappedBy = "user",
74:             cascade = {CascadeType.PERSIST, CascadeType.MERGE},
75:             orphanRemoval = true)
76:     private Set<Product> products;
77: }
```

File Path: src/main/java/com/ecommerce/project/payload/APIResponse.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'APIResponse' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class APIResponse {
11:     public String message;
12:     private boolean status;
13: }
```

File Path: src/main/java/com/ecommerce/project/payload/AddressDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'AddressDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class AddressDTO {
11:     private Long addressId;
12:     private String street;
13:     private String buildingName;
14:     private String city;
15:     private String state;
16:     private String country;
17:     private String pincode;
18: }
```

File Path: src/main/java/com/ecommerce/project/payload/CartDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'CartDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: import java.util.ArrayList;
 8: import java.util.List;
 9:
10: @Data
11: @NoArgsConstructor
12: @AllArgsConstructor
13: public class CartDTO {
14:     private Long cartId;
15:     private Double totalPrice = 0.0;
16:     private List<ProductDTO> products = new ArrayList<>();
17: }
```

File Path: src/main/java/com/ecommerce/project/payload/CartItemDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*
Injected Dependencies:	cart (CartDTO), productDTO (ProductDTO)

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'CartItemDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class CartItemDTO {
11:     private Long cartItemId;
12:     private CartDTO cart;
13:     private ProductDTO productDTO;
14:     private Integer quantity;
15:     private Double discount;
16:     private Double productPrice;
17: }
```

File Path: src/main/java/com/ecommerce/project/payload/CategoryDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'CategoryDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class CategoryDTO {
11:     private Long categoryId;
12:     private String categoryName;
13: }
```

File Path: src/main/java/com/ecommerce/project/payload/CategoryResponse.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'CategoryResponse' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: import java.util.List;
 8:
 9: @Data
10: @NoArgsConstructor
11: @AllArgsConstructor
12: public class CategoryResponse {
13:     private List<CategoryDTO> content;
14:     private Integer pageNumber;
15:     private Integer pageSize;
16:     private Long totalElements;
17:     private Integer totalPages;
18:     private boolean lastPage;
19: }
```

File Path: src/main/java/com/ecommerce/project/payload/OrderDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*
Injected Dependencies:	orderDate (LocalDate), payment (PaymentDTO)

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'OrderDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: import java.time.LocalDate;
 8: import java.util.List;
 9:
10: @Data
11: @NoArgsConstructor
12: @AllArgsConstructor
13: public class OrderDTO {
14:     private Long orderId;
15:     private String email;
16:     private List<OrderItemDTO> orderItems;
17:     private LocalDate orderDate;
18:     private PaymentDTO payment;
19:     private Double totalAmount;
20:     private String orderStatus;
21:     private Long addressId;
22: }
```

File Path: src/main/java/com/ecommerce/project/payload/OrderItemDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*
Injected Dependencies:	product (ProductDTO)

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'OrderItemDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class OrderItemDTO {
11:     private Long orderItemId;
12:     private ProductDTO product;
13:     private Integer quantity;
14:     private double discount;
15:     private double orderedProductPrice;
16: }
```

File Path: src/main/java/com/ecommerce/project/payload/OrderRequestDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**

Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'OrderRequestDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class OrderRequestDTO {
11:     private Long addressId;
12:     private String paymentMethod;
13:     private String pgName;
14:     private String pgPaymentId;
15:     private String pgStatus;
16:     private String pgResponseMessage;
17: }
```

File Path: src/main/java/com/ecommerce/project/payload/PaymentDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'PaymentDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class PaymentDTO {
11:     private Long paymentId;
12:     private String paymentMethod;
13:     private String pgPaymentId;
14:     private String pgStatus;
15:     private String pgResponseMessage;
16:     private String pgName;
17: }
```

File Path: src/main/java/com/ecommerce/project/payload/ProductDTO.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'ProductDTO' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: @Data
 8: @NoArgsConstructor
 9: @AllArgsConstructor
10: public class ProductDTO {
11:     private Long productId;
12:     private String productName;
13:     private String image;
14:     private String description;
15:     private Integer quantity;
16:     private double price;
17:     private double discount;
18:     private double specialPrice;
19: }
```

File Path: src/main/java/com/ecommerce/project/payload/ProductResponse.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'ProductResponse' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.payload;
 2:
 3: import lombok.AllArgsConstructor;
 4: import lombok.Data;
 5: import lombok.NoArgsConstructor;
 6:
 7: import java.util.List;
 8:
 9: @Data
10: @NoArgsConstructor
11: @AllArgsConstructor
12: public class ProductResponse {
13:     private List<ProductDTO> content;
14:     private Integer pageNumber;
15:     private Integer pageSize;
16:     private Long totalElements;
17:     private Integer totalPages;
18:     private boolean lastPage;
19: }
```

File Path: src/main/java/com/ecommerce/project/repositories/AddressRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**

Technical Explanation of Code Logic:

Interface for database execution on 'Address' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
1: package com.ecommerce.project.repositories;
2:
3: import com.ecommerce.project.model.Address;
4: import org.springframework.data.jpa.repository.JpaRepository;
5:
6: public interface AddressRepository extends JpaRepository<Address, Long> {
7: }
```

File Path: src/main/java/com/ecommerce/project/repositories/CartItemRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Modifying, @Query*

Technical Explanation of Code Logic:

Interface for database execution on 'CartItem' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3: import com.ecommerce.project.model.CartItem;
 4: import org.springframework.data.jpa.repository.JpaRepository;
 5: import org.springframework.data.jpa.repository.Modifying;
 6: import org.springframework.data.jpa.repository.Query;
 7:
 8: public interface CartItemRepository extends JpaRepository<CartItem, Long> {
 9:     @Query("SELECT ci FROM CartItem ci WHERE ci.cart.id = ?1 AND ci.product.id = ?2")
10:     CartItem findCartItemByProductIdAndCartId(Long cartId, Long productId);
11:
12:     @Modifying
13:     @Query("DELETE FROM CartItem ci WHERE ci.cart.id = ?1 AND ci.product.id = ?2")
14:     void deleteCartItemByProductIdAndCartId(Long cartId, Long productId);
15: }
```

File Path: src/main/java/com/ecommerce/project/repositories/CartRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Query*

Technical Explanation of Code Logic:

Interface for database execution on 'Cart' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
1: package com.ecommerce.project.repositories;
2:
3: import com.ecommerce.project.model.Cart;
4: import org.springframework.data.jpa.repository.JpaRepository;
```

```
 5: import org.springframework.data.jpa.repository.Query;
 6:
 7: import java.util.List;
 8:
 9: public interface CartRepository extends JpaRepository<Cart, Long> {
10:     @Query("SELECT c FROM Cart c WHERE c.user.email = ?1")
11:     Cart findCartByEmail(String email);
12:
13:     @Query("SELECT c FROM Cart c WHERE c.user.email = ?1 AND c.id = ?2")
14:     Cart findCartByEmailAndCartId(String emailId, Long cartId);
15:
16:     @Query("SELECT c FROM Cart c JOIN FETCH c.cartItems ci JOIN FETCH ci.product p WHERE p.id =
    ?1")
17:     List<Cart> findCartsByProductId(Long productId);
18: }
```

File Path: src/main/java/com/ecommerce/project/repositories/CategoryRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**

Technical Explanation of Code Logic:

Interface for database execution on 'Category' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
1: package com.ecommerce.project.repositories;
2:
3: import com.ecommerce.project.model.Category;
4: import org.springframework.data.jpa.repository.JpaRepository;
5:
6: public interface CategoryRepository extends JpaRepository<Category,Long> {
7:     Category findByCategoryName(String categoryName);
8: }
```

File Path: src/main/java/com/ecommerce/project/repositories/OrderItemRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Repository*

Technical Explanation of Code Logic:

Interface for database execution on 'OrderItem' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3:
 4: import org.springframework.data.jpa.repository.JpaRepository;
 5: import org.springframework.stereotype.Repository;
 6:
 7: import com.ecommerce.project.model.OrderItem;
 8:
 9: @Repository
10: public interface OrderItemRepository extends JpaRepository<OrderItem, Long> {
11:
```

```
12: }
```

File Path: src/main/java/com/ecommerce/project/repositories/OrderRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Repository*

Technical Explanation of Code Logic:

Interface for database execution on 'Order' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3:
 4: import org.springframework.data.jpa.repository.JpaRepository;
 5: import org.springframework.stereotype.Repository;
 6:
 7: import com.ecommerce.project.model.Order;
 8:
 9: @Repository
10: public interface OrderRepository extends JpaRepository<Order, Long> {
11:
12: }
```

File Path: src/main/java/com/ecommerce/project/repositories/PaymentRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Repository*

Technical Explanation of Code Logic:

Interface for database execution on 'Payment' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3:
 4: import org.springframework.data.jpa.repository.JpaRepository;
 5: import org.springframework.stereotype.Repository;
 6:
 7: import com.ecommerce.project.model.Payment;
 8:
 9: @Repository
10: public interface PaymentRepository extends JpaRepository<Payment, Long>{
11:
12: }
```

File Path: src/main/java/com/ecommerce/project/repositories/ProductRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Repository*

Technical Explanation of Code Logic:

Interface for database execution on 'Product' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3: import com.ecommerce.project.model.Category;
 4: import com.ecommerce.project.model.Product;
 5: import org.springframework.data.domain.Page;
 6: import org.springframework.data.domain.Pageable;
 7: import org.springframework.data.jpa.repository.JpaRepository;
 8: import org.springframework.stereotype.Repository;
 9:
10: @Repository
11: public interface ProductRepository extends JpaRepository<Product, Long> {
12:     Page<Product> findByCategoryOrderByPriceAsc(Category category, Pageable pageDetails);
13:
14:     Page<Product> findByProductNameLikeIgnoreCase(String keyword, Pageable pageDetails);
15: }
```

File Path: src/main/java/com/ecommerce/project/repositories/RoleRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**

Technical Explanation of Code Logic:

Interface for database execution on 'Role' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3: import com.ecommerce.project.model.AppRole;
 4: import com.ecommerce.project.model.Role;
 5: import org.springframework.data.jpa.repository.JpaRepository;
 6:
 7: import java.util.Optional;
 8:
 9: public interface RoleRepository extends JpaRepository<Role, Long> {
10:     Optional<Role> findByRoleName(AppRole appRole);
11: }
```

File Path: src/main/java/com/ecommerce/project/repositories/UserRepository.java

COMPONENT PROFILE

Stereotype:	**JPA Repository (Database Queries)**
Active Annotations:	*@Repository*

Technical Explanation of Code Logic:

Interface for database execution on 'User' entity objects. Inherits standard database transactions from Spring Data's JpaRepository.

Source Code Listing:

```
 1: package com.ecommerce.project.repositories;
 2:
 3: import com.ecommerce.project.model.User;
 4: import org.springframework.data.jpa.repository.JpaRepository;
 5: import org.springframework.stereotype.Repository;
 6:
 7: import java.util.Optional;
 8:
 9: @Repository
10: public interface UserRepository extends JpaRepository<User, Long> {
11:
12:     Optional<User> findByUserName(String username);
13:
14:     Boolean existsByUserName(String username);
15:
16:     Boolean existsByEmail(String email);
17: }
```

File Path: src/main/java/com/ecommerce/project/security/WebSecurityConfig.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**
Active Annotations:	*@Bean, @Configuration, @EnableMethodSecurity, @EnableWebSecurity, @example*
Injected Dependencies:	unauthorizedHandler (AuthEntryPointJwt)

Technical Explanation of Code Logic:

WebSecurityConfig configures the Spring Security filter chain. It disables CSRF (since we use stateless cookies/tokens), specifies unauthorized access handlers, configures endpoint authorization rules (allowing public access to auth/category endpoints but requiring authentication for cart/checkout), and registers the AuthTokenFilter.

Source Code Listing:

```
 1: package com.ecommerce.project.security;
 2:
 3: import com.ecommerce.project.model.AppRole;
 4: import com.ecommerce.project.model.Role;
 5: import com.ecommerce.project.model.User;
 6: import com.ecommerce.project.repositories.RoleRepository;
 7: import com.ecommerce.project.repositories.UserRepository;
 8: import org.springframework.beans.factory.annotation.Autowired;
 9: import org.springframework.boot.CommandLineRunner;
10: import org.springframework.context.annotation.Bean;
11: import org.springframework.context.annotation.Configuration;
12: import org.springframework.security.authentication.AuthenticationManager;
13: import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
14: //import org.springframework.security.config.annotation.authentication.builders.AuthenticationM
    anagerBuilder;
15: import org.springframework.security.config.annotation.authentication.configuration.Authenticati
    onConfiguration;
16: import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity
    ;
17: import org.springframework.security.config.annotation.web.builders.HttpSecurity;
18: //import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
19: //import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurer
    Adapter;
20: import org.springframework.security.config.annotation.web.builders.WebSecurity;
21: import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
22: import org.springframework.security.config.annotation.web.configuration.WebSecurityCustomizer;
23: import org.springframework.security.config.http.SessionCreationPolicy;
24: import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
25: import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

import com.ecommerce.project.security.jwt.AuthEntryPointJwt;
import com.ecommerce.project.security.jwt.AuthTokenFilter;
import com.ecommerce.project.security.services.UserDetailsServiceImpl;

import java.util.Set;

@Configuration
@EnableWebSecurity
//@EnableMethodSecurity
public class WebSecurityConfig {
    @Autowired
    UserDetailsServiceImpl userDetailsService;

    @Autowired
    private AuthEntryPointJwt unauthorizedHandler;

    @Bean
    public AuthTokenFilter authenticationJwtTokenFilter() {
        return new AuthTokenFilter();
    }


    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();

        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());

        return authProvider;
    }


    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig)
    throws Exception {
        return authConfig.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }



    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
                .exceptionHandling(exception -> exception.authenticationEntryPoint(unauthorized
    Handler))
                .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPoli
    cy.STATELESS))
                .authorizeHttpRequests(auth ->
                        auth.requestMatchers("/api/auth/**").permitAll()
                                .requestMatchers("/v3/api-docs/**").permitAll()
                                .requestMatchers("/h2-console/**").permitAll()
                                //.requestMatchers("/api/admin/**").permitAll()
                                //.requestMatchers("/api/public/**").permitAll()
                                .requestMatchers("/swagger-ui/**").permitAll()
                                .requestMatchers("/api/test/**").permitAll()
                                .requestMatchers("/images/**").permitAll()
                                .anyRequest().authenticated()
                );

        http.authenticationProvider(authenticationProvider());

        http.addFilterBefore(authenticationJwtTokenFilter(), UsernamePasswordAuthenticationFilt
    er.class);
        http.headers(headers -> headers.frameOptions(
                frameOptions -> frameOptions.sameOrigin()));

        return http.build();
    }

    @Bean
    public WebSecurityCustomizer webSecurityCustomizer() {
```

```
        return (web -> web.ignoring().requestMatchers("/v2/api-docs",
                "/configuration/ui",
                "/swagger-resources/**",
                "/configuration/security",
                "/swagger-ui.html",
                "/webjars/**"));
    }


    @Bean
    public CommandLineRunner initData(RoleRepository roleRepository, UserRepository userReposit
ory, PasswordEncoder passwordEncoder) {
        return args -> {
            // Retrieve or create roles
            Role userRole = roleRepository.findByRoleName(AppRole.ROLE_USER)
                    .orElseGet(() -> {
                        Role newUserRole = new Role(AppRole.ROLE_USER);
                        return roleRepository.save(newUserRole);
                    });

            Role sellerRole = roleRepository.findByRoleName(AppRole.ROLE_SELLER)
                    .orElseGet(() -> {
                        Role newSellerRole = new Role(AppRole.ROLE_SELLER);
                        return roleRepository.save(newSellerRole);
                    });

            Role adminRole = roleRepository.findByRoleName(AppRole.ROLE_ADMIN)
                    .orElseGet(() -> {
                        Role newAdminRole = new Role(AppRole.ROLE_ADMIN);
                        return roleRepository.save(newAdminRole);
                    });

            Set<Role> userRoles = Set.of(userRole);
            Set<Role> sellerRoles = Set.of(sellerRole);
            Set<Role> adminRoles = Set.of(userRole, sellerRole, adminRole);


            // Create users if not already present
            if (!userRepository.existsByUserName("user1")) {
                User user1 = new User("user1", "user1@example.com", passwordEncoder.encode("pas
sword1"));
                userRepository.save(user1);
            }

            if (!userRepository.existsByUserName("seller1")) {
                User seller1 = new User("seller1", "seller1@example.com", passwordEncoder.encod
e("password2"));
                userRepository.save(seller1);
            }

            if (!userRepository.existsByUserName("admin")) {
                User admin = new User("admin", "admin@example.com", passwordEncoder.encode("adm
inPass"));
                userRepository.save(admin);
            }

            // Update roles for existing users
            userRepository.findByUserName("user1").ifPresent(user -> {
                user.setRoles(userRoles);
                userRepository.save(user);
            });

            userRepository.findByUserName("seller1").ifPresent(seller -> {
                seller.setRoles(sellerRoles);
                userRepository.save(seller);
            });

            userRepository.findByUserName("admin").ifPresent(admin -> {
                admin.setRoles(adminRoles);
                userRepository.save(admin);
            });
        };
    }

}
```

File Path: src/main/java/com/ecommerce/project/security/jwt/AuthEntryPointJwt.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**
Active Annotations:	*@Component*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'AuthEntryPointJwt' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.security.jwt;
 2:
 3: import com.fasterxml.jackson.databind.ObjectMapper;
 4: import jakarta.servlet.ServletException;
 5: import jakarta.servlet.http.HttpServletRequest;
 6: import jakarta.servlet.http.HttpServletResponse;
 7: import org.slf4j.Logger;
 8: import org.slf4j.LoggerFactory;
 9: import org.springframework.http.MediaType;
10: import org.springframework.security.core.AuthenticationException;
11: import org.springframework.security.web.AuthenticationEntryPoint;
12: import org.springframework.stereotype.Component;
13:
14: import java.io.IOException;
15: import java.util.HashMap;
16: import java.util.Map;
17:
18: @Component
19: public class AuthEntryPointJwt implements AuthenticationEntryPoint {
20:
21:     private static final Logger logger = LoggerFactory.getLogger(AuthEntryPointJwt.class);
22:
23:     @Override
24:     public void commence(HttpServletRequest request, HttpServletResponse response, Authenticati
    onException authException)
25:             throws IOException, ServletException {
26:         logger.error("Unauthorized error: {}", authException.getMessage());
27:
28:         response.setContentType(MediaType.APPLICATION_JSON_VALUE);
29:         response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
30:
31:         final Map<String, Object> body = new HashMap<>();
32:         body.put("status", HttpServletResponse.SC_UNAUTHORIZED);
33:         body.put("error", "Unauthorized");
34:         body.put("message", authException.getMessage());
35:         body.put("path", request.getServletPath());
36:
37:         final ObjectMapper mapper = new ObjectMapper();
38:         mapper.writeValue(response.getOutputStream(), body);
39:     }
40:
41: }
42:
```

File Path: src/main/java/com/ecommerce/project/security/jwt/AuthTokenFilter.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**
Active Annotations:	*@Component*
Injected Dependencies:	jwtUtils (JwtUtils), userDetailsService (UserDetailsServiceImpl)

Technical Explanation of Code Logic:

AuthTokenFilter is a custom OncePerRequestFilter that intercept every incoming HTTP request. It extracts the JWT from the cookie

(or authorization headers), validates the token via JwtUtils, loads user details using UserDetailsService, and registers the user in Spring Security's SecurityContextHolder. This enables stateless, secure page access.

Source Code Listing:

```
 1: package com.ecommerce.project.security.jwt;
 2:
 3: import com.ecommerce.project.security.services.UserDetailsServiceImpl;
 4: import jakarta.servlet.FilterChain;
 5: import jakarta.servlet.ServletException;
 6: import jakarta.servlet.http.HttpServletRequest;
 7: import jakarta.servlet.http.HttpServletResponse;
 8: import org.slf4j.Logger;
 9: import org.slf4j.LoggerFactory;
10: import org.springframework.beans.factory.annotation.Autowired;
11: import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
12: import org.springframework.security.core.context.SecurityContextHolder;
13: import org.springframework.security.core.userdetails.UserDetails;
14: import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
15: import org.springframework.stereotype.Component;
16: import org.springframework.web.filter.OncePerRequestFilter;
17:
18: import java.io.IOException;
19:
20: @Component
21: public class AuthTokenFilter extends OncePerRequestFilter {
22:     @Autowired
23:     private JwtUtils jwtUtils;
24:
25:     @Autowired
26:     private UserDetailsServiceImpl userDetailsService;
27:
28:     private static final Logger logger = LoggerFactory.getLogger(AuthTokenFilter.class);
29:
30:     @Override
31:     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, F
    ilterChain filterChain)
32:             throws ServletException, IOException {
33:         logger.debug("AuthTokenFilter called for URI: {}", request.getRequestURI());
34:         try {
35:             String jwt = parseJwt(request);
36:             if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
37:                 String username = jwtUtils.getUserNameFromJwtToken(jwt);
38:
39:                 UserDetails userDetails = userDetailsService.loadUserByUsername(username);
40:
41:                 UsernamePasswordAuthenticationToken authentication =
42:                         new UsernamePasswordAuthenticationToken(userDetails,
43:                                 null,
44:                                 userDetails.getAuthorities());
45:                 logger.debug("Roles from JWT: {}", userDetails.getAuthorities());
46:
47:                 authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(req
    uest));
48:
49:                 SecurityContextHolder.getContext().setAuthentication(authentication);
50:             }
51:         } catch (Exception e) {
52:             logger.error("Cannot set user authentication: {}", e);
53:         }
54:
55:         filterChain.doFilter(request, response);
56:     }
57:
58:     private String parseJwt(HttpServletRequest request) {
59:         String jwt = jwtUtils.getJwtFromCookies(request);
60:         logger.debug("AuthTokenFilter.java: {}", jwt);
61:         return jwt;
62:     }
63: }
64:
```

File Path: src/main/java/com/ecommerce/project/security/jwt/JwtUtils.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**
Active Annotations:	*@Component, @Value*

Technical Explanation of Code Logic:

JwtUtils is a helper class for working with JSON Web Tokens (JWT). It generates a token containing the username, signs it using a secure HMAC key, extracts usernames from tokens, validates token signatures and expiration, and manages cookie generation for embedding the JWT securely in client browsers.

Source Code Listing:

```
 1: package com.ecommerce.project.security.jwt;
 2:
 3: import com.ecommerce.project.security.services.UserDetailsImpl;
 4: import io.jsonwebtoken.*;
 5: import io.jsonwebtoken.io.Decoders;
 6: import io.jsonwebtoken.security.Keys;
 7: import jakarta.servlet.http.Cookie;
 8: import jakarta.servlet.http.HttpServletRequest;
 9: import org.slf4j.Logger;
10: import org.slf4j.LoggerFactory;
11: import org.springframework.beans.factory.annotation.Value;
12: import org.springframework.http.ResponseCookie;
13: import org.springframework.security.core.userdetails.UserDetails;
14: import org.springframework.stereotype.Component;
15: import org.springframework.web.util.WebUtils;
16:
17: import javax.crypto.SecretKey;
18: import java.security.Key;
19: import java.util.Date;
20:
21: @Component
22: public class JwtUtils {
23:     private static final Logger logger = LoggerFactory.getLogger(JwtUtils.class);
24:
25:     @Value("${spring.app.jwtSecret}")
26:     private String jwtSecret;
27:
28:     @Value("${spring.app.jwtExpirationMs}")
29:     private int jwtExpirationMs;
30:
31:     @Value("${spring.ecom.app.jwtCookieName}")
32:     private String jwtCookie;
33:
34:     public String getJwtFromCookies(HttpServletRequest request) {
35:         Cookie cookie = WebUtils.getCookie(request, jwtCookie);
36:         if (cookie != null) {
37:             return cookie.getValue();
38:         } else {
39:             return null;
40:         }
41:     }
42:
43:     public ResponseCookie generateJwtCookie(UserDetailsImpl userPrincipal) {
44:         String jwt = generateTokenFromUsername(userPrincipal.getUsername());
45:         ResponseCookie cookie = ResponseCookie.from(jwtCookie, jwt)
46:                 .path("/api")
47:                 .maxAge(24 * 60 * 60)
48:                 .httpOnly(false)
49:                 .build();
50:         return cookie;
51:     }
52:
53:     public ResponseCookie getCleanJwtCookie() {
54:         ResponseCookie cookie = ResponseCookie.from(jwtCookie, null)
55:                 .path("/api")
56:                 .build();
57:         return cookie;
58:     }
59:
60:     public String generateTokenFromUsername(String username) {
61:         return Jwts.builder()
```

```
62:                 .subject(username)
63:                 .issuedAt(new Date())
64:                 .expiration(new Date((new Date()).getTime() + jwtExpirationMs))
65:                 .signWith(key())
66:                 .compact();
67:     }
68:
69:     public String getUserNameFromJwtToken(String token) {
70:         return Jwts.parser()
71:                         .verifyWith((SecretKey) key())
72:                 .build().parseSignedClaims(token)
73:                 .getPayload().getSubject();
74:     }
75:
76:     private Key key() {
77:         return Keys.hmacShaKeyFor(Decoders.BASE64.decode(jwtSecret));
78:     }
79:
80:     public boolean validateJwtToken(String authToken) {
81:         try {
82:             Jwts.parser().verifyWith((SecretKey) key()).build().parseSignedClaims(authToken);
83:             return true;
84:         } catch (MalformedJwtException e) {
85:             logger.error("Invalid JWT token: {}", e.getMessage());
86:         } catch (ExpiredJwtException e) {
87:             logger.error("JWT token is expired: {}", e.getMessage());
88:         } catch (UnsupportedJwtException e) {
89:             logger.error("JWT token is unsupported: {}", e.getMessage());
90:         } catch (IllegalArgumentException e) {
91:             logger.error("JWT claims string is empty: {}", e.getMessage());
92:         }
93:         return false;
94:     }
95: }
```

File Path: src/main/java/com/ecommerce/project/security/request/LoginRequest.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@NotBlank*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'LoginRequest' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.security.request;
 2:
 3: import jakarta.validation.constraints.NotBlank;
 4:
 5: public class LoginRequest {
 6:     @NotBlank
 7:     private String username;
 8:
 9:     @NotBlank
10:     private String password;
11:
12:     public String getUsername() {
13:         return username;
14:     }
15:
16:     public void setUsername(String username) {
17:         this.username = username;
18:     }
19:
20:     public String getPassword() {
21:         return password;
22:     }
23:
24:     public void setPassword(String password) {
```

```
25:         this.password = password;
26:     }
27: }
```

File Path: src/main/java/com/ecommerce/project/security/request/SignupRequest.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**
Active Annotations:	*@Data, @Email, @NotBlank, @Size*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'SignupRequest' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.security.request;
 2:
 3: import java.util.Set;
 4:
 5: import jakarta.validation.constraints.*;
 6: import lombok.Data;
 7:
 8: @Data
 9: public class SignupRequest {
10:     @NotBlank
11:     @Size(min = 3, max = 20)
12:     private String username;
13:
14:     @NotBlank
15:     @Size(max = 50)
16:     @Email
17:     private String email;
18:
19:     private Set<String> role;
20:
21:     @NotBlank
22:     @Size(min = 6, max = 40)
23:     private String password;
24:
25:     public Set<String> getRole() {
26:         return this.role;
27:     }
28:
29:     public void setRole(Set<String> role) {
30:         this.role = role;
31:     }
32: }
```

File Path: src/main/java/com/ecommerce/project/security/response/MessageResponse.java

COMPONENT PROFILE

Stereotype:	**Data Transfer Object (DTO)**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'MessageResponse' workflow executing smoothly.

Source Code Listing:

```
1: package com.ecommerce.project.security.response;
2:
```

```
 3: public class MessageResponse {
 4:     private String message;
 5:
 6:     public MessageResponse(String message) {
 7:         this.message = message;
 8:     }
 9:
10:     public String getMessage() {
11:         return message;
12:     }
13:
14:     public void setMessage(String message) {
15:         this.message = message;
16:     }
17: }
```

File Path: src/main/java/com/ecommerce/project/security/response/UserInfoResponse.java

COMPONENT PROFILE

Stereotype: **Data Transfer Object (DTO)**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'UserInfoResponse' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.security.response;
 2:
 3: import java.util.List;
 4:
 5: public class UserInfoResponse {
 6:     private Long id;
 7:     private String jwtToken;
 8:     private String username;
 9:     private List<String> roles;
10:
11:     public UserInfoResponse(Long id, String username, List<String> roles, String jwtToken) {
12:         this.id = id;
13:         this.username = username;
14:         this.roles = roles;
15:         this.jwtToken = jwtToken;
16:     }
17:
18:     public UserInfoResponse(Long id, String username, List<String> roles) {
19:         this.id = id;
20:         this.username = username;
21:         this.roles = roles;
22:     }
23:
24:     public Long getId() {
25:         return id;
26:     }
27:
28:     public void setId(Long id) {
29:         this.id = id;
30:     }
31:
32:     public String getJwtToken() {
33:         return jwtToken;
34:     }
35:
36:     public void setJwtToken(String jwtToken) {
37:         this.jwtToken = jwtToken;
38:     }
39:
40:     public String getUsername() {
41:         return username;
42:     }
43:
44:     public void setUsername(String username) {
```

```
45:         this.username = username;
46:     }
47:
48:     public List<String> getRoles() {
49:         return roles;
50:     }
51:
52:     public void setRoles(List<String> roles) {
53:         this.roles = roles;
54:     }
55: }
56:
57:
```

File Path: src/main/java/com/ecommerce/project/security/services/UserDetailsImpl.java

COMPONENT PROFILE

Stereotype:	**Security Config / Middleware Filter**
Active Annotations:	*@Data, @JsonIgnore*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'UserDetailsImpl' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.security.services;
 2:
 3: import java.util.Collection;
 4: import java.util.List;
 5: import java.util.Objects;
 6: import java.util.stream.Collectors;
 7:
 8: import lombok.Data;
 9: import lombok.NoArgsConstructor;
10: import org.springframework.security.core.GrantedAuthority;
11: import org.springframework.security.core.authority.SimpleGrantedAuthority;
12: import org.springframework.security.core.userdetails.UserDetails;
13:
14: import com.ecommerce.project.model.User;
15: import com.fasterxml.jackson.annotation.JsonIgnore;
16:
17: @NoArgsConstructor
18: @Data
19: public class UserDetailsImpl implements UserDetails {
20:     private static final long serialVersionUID = 1L;
21:
22:     private Long id;
23:
24:     private String username;
25:
26:     private String email;
27:
28:     @JsonIgnore
29:     private String password;
30:
31:     private Collection<? extends GrantedAuthority> authorities;
32:
33:     public UserDetailsImpl(Long id, String username, String email, String password,
34:                            Collection<? extends GrantedAuthority> authorities) {
35:         this.id = id;
36:         this.username = username;
37:         this.email = email;
38:         this.password = password;
39:         this.authorities = authorities;
40:     }
41:
42:     public static UserDetailsImpl build(User user) {
43:         List<GrantedAuthority> authorities = user.getRoles().stream()
44:                 .map(role -> new SimpleGrantedAuthority(role.getRoleName().name()))
45:                 .collect(Collectors.toList());
```

```
46:
47:         return new UserDetailsImpl(
48:                 user.getUserId(),
49:                 user.getUserName(),
50:                 user.getEmail(),
51:                 user.getPassword(),
52:                 authorities);
53:     }
54:
55:     @Override
56:     public Collection<? extends GrantedAuthority> getAuthorities() {
57:         return authorities;
58:     }
59:
60:     public Long getId() {
61:         return id;
62:     }
63:
64:     public String getEmail() {
65:         return email;
66:     }
67:
68:     @Override
69:     public String getPassword() {
70:         return password;
71:     }
72:
73:     @Override
74:     public String getUsername() {
75:         return username;
76:     }
77:
78:     @Override
79:     public boolean isAccountNonExpired() {
80:         return true;
81:     }
82:
83:     @Override
84:     public boolean isAccountNonLocked() {
85:         return true;
86:     }
87:
88:     @Override
89:     public boolean isCredentialsNonExpired() {
90:         return true;
91:     }
92:
93:     @Override
94:     public boolean isEnabled() {
95:         return true;
96:     }
97:
98:     @Override
99:     public boolean equals(Object o) {
100:         if (this == o)
101:             return true;
102:         if (o == null || getClass() != o.getClass())
103:             return false;
104:         UserDetailsImpl user = (UserDetailsImpl) o;
105:         return Objects.equals(id, user.id);
106:     }
107:
108: }
```

File Path: src/main/java/com/ecommerce/project/security/services/UserDetailsServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service, @Transactional*

Technical Explanation of Code Logic:

UserDetailsServiceImpl implements UserDetailsService, which is called by Spring Security to load a user from the database by username. It returns a UserDetailsImpl object populated with the user's credentials and authorities.

Source Code Listing:

```
 1: package com.ecommerce.project.security.services;
 2:
 3: import org.springframework.beans.factory.annotation.Autowired;
 4: import org.springframework.security.core.Authentication;
 5: import org.springframework.security.core.context.SecurityContextHolder;
 6: import org.springframework.security.core.userdetails.UserDetails;
 7: import org.springframework.security.core.userdetails.UserDetailsService;
 8: import org.springframework.security.core.userdetails.UsernameNotFoundException;
 9: import org.springframework.stereotype.Service;
10: import org.springframework.transaction.annotation.Transactional;
11:
12: import com.ecommerce.project.model.User;
13: import com.ecommerce.project.repositories.UserRepository;
14:
15: @Service
16: public class UserDetailsServiceImpl implements UserDetailsService {
17:     @Autowired
18:     UserRepository userRepository;
19:
20:     @Override
21:     @Transactional
22:     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
23:         User user = userRepository.findByUserName(username)
24:                 .orElseThrow(() -> new UsernameNotFoundException("User Not Found with username:
     " + username));
25:
26:         return UserDetailsImpl.build(user);
27:     }
28:
29:
30: }
```

File Path: src/main/java/com/ecommerce/project/service/AddressService.java

COMPONENT PROFILE

Stereotype: **Service Interface**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'AddressService' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.model.User;
 4: import com.ecommerce.project.payload.AddressDTO;
 5:
 6: import java.util.List;
 7:
 8: public interface AddressService {
 9:     AddressDTO createAddress(AddressDTO addressDTO, User user);
10:
11:     List<AddressDTO> getAddresses();
12:
13:     AddressDTO getAddressesById(Long addressId);
14:
15:     List<AddressDTO> getUserAddresses(User user);
16:
17:     AddressDTO updateAddress(Long addressId, AddressDTO addressDTO);
18:
19:     String deleteAddress(Long addressId);
20: }
```

File Path: src/main/java/com/ecommerce/project/service/AddressServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service*
Injected Dependencies:	addressRepository (AddressRepository), modelMapper (ModelMapper)

Technical Explanation of Code Logic:

Executes key transaction rules and calculations for 'Address'. It handles validation thresholds, queries database records, and mappings configurations.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.exceptions.ResourceNotFoundException;
 4: import com.ecommerce.project.model.Address;
 5: import com.ecommerce.project.model.User;
 6: import com.ecommerce.project.payload.AddressDTO;
 7: import com.ecommerce.project.repositories.AddressRepository;
 8: import com.ecommerce.project.repositories.UserRepository;
 9: import org.modelmapper.ModelMapper;
10: import org.springframework.beans.factory.annotation.Autowired;
11: import org.springframework.stereotype.Service;
12:
13: import java.util.List;
14:
15: @Service
16: public class AddressServiceImpl implements AddressService{
17:     @Autowired
18:     private AddressRepository addressRepository;
19:
20:     @Autowired
21:     private ModelMapper modelMapper;
22:
23:     @Autowired
24:     UserRepository userRepository;
25:
26:     @Override
27:     public AddressDTO createAddress(AddressDTO addressDTO, User user) {
28:         Address address = modelMapper.map(addressDTO, Address.class);
29:         address.setUser(user);
30:         List<Address> addressesList = user.getAddresses();
31:         addressesList.add(address);
32:         user.setAddresses(addressesList);
33:         Address savedAddress = addressRepository.save(address);
34:         return modelMapper.map(savedAddress, AddressDTO.class);
35:     }
36:
37:     @Override
38:     public List<AddressDTO> getAddresses() {
39:         List<Address> addresses = addressRepository.findAll();
40:         return addresses.stream()
41:                 .map(address -> modelMapper.map(address, AddressDTO.class))
42:                 .toList();
43:     }
44:
45:     @Override
46:     public AddressDTO getAddressesById(Long addressId) {
47:         Address address = addressRepository.findById(addressId)
48:                 .orElseThrow(() -> new ResourceNotFoundException("Address", "addressId", addres
    sId));
49:         return modelMapper.map(address, AddressDTO.class);
50:     }
51:
52:     @Override
53:     public List<AddressDTO> getUserAddresses(User user) {
54:         List<Address> addresses = user.getAddresses();
55:         return addresses.stream()
56:                 .map(address -> modelMapper.map(address, AddressDTO.class))
57:                 .toList();
58:     }
59:
60:     @Override
```

```
    public AddressDTO updateAddress(Long addressId, AddressDTO addressDTO) {
        Address addressFromDatabase = addressRepository.findById(addressId)
                .orElseThrow(() -> new ResourceNotFoundException("Address", "addressId", addres
sId));

        addressFromDatabase.setCity(addressDTO.getCity());
        addressFromDatabase.setPincode(addressDTO.getPincode());
        addressFromDatabase.setState(addressDTO.getState());
        addressFromDatabase.setCountry(addressDTO.getCountry());
        addressFromDatabase.setStreet(addressDTO.getStreet());
        addressFromDatabase.setBuildingName(addressDTO.getBuildingName());

        Address updatedAddress = addressRepository.save(addressFromDatabase);

        User user = addressFromDatabase.getUser();
        user.getAddresses().removeIf(address -> address.getAddressId().equals(addressId));
        user.getAddresses().add(updatedAddress);
        userRepository.save(user);

        return modelMapper.map(updatedAddress, AddressDTO.class);
    }

    @Override
    public String deleteAddress(Long addressId) {
        Address addressFromDatabase = addressRepository.findById(addressId)
                .orElseThrow(() -> new ResourceNotFoundException("Address", "addressId", addres
sId));

        User user = addressFromDatabase.getUser();
        user.getAddresses().removeIf(address -> address.getAddressId().equals(addressId));
        userRepository.save(user);

        addressRepository.delete(addressFromDatabase);

        return "Address deleted successfully with addressId: " + addressId;
    }
}
```

File Path: src/main/java/com/ecommerce/project/service/CartService.java

COMPONENT PROFILE

Stereotype:	**Service Interface**
Active Annotations:	*@Transactional*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'CartService' workflow executing smoothly.

Source Code Listing:

```
package com.ecommerce.project.service;

import com.ecommerce.project.payload.CartDTO;
import jakarta.transaction.Transactional;

import java.util.List;

public interface CartService {
    CartDTO addProductToCart(Long productId, Integer quantity);

    List<CartDTO> getAllCarts();

    CartDTO getCart(String emailId, Long cartId);

    @Transactional
    CartDTO updateProductQuantityInCart(Long productId, Integer quantity);

    String deleteProductFromCart(Long cartId, Long productId);

    void updateProductInCarts(Long cartId, Long productId);
}
```

File Path: src/main/java/com/ecommerce/project/service/CartServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service, @Transactional*
Injected Dependencies:	cartRepository (CartRepository), authUtil (AuthUtil)

Technical Explanation of Code Logic:

CartServiceImpl implements shopping cart logic. It retrieves/creates carts, validates product stock availability, applies discounts to calculate special pricing, handles cart merge/updates, and cleans up cart instances after checkout.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.exceptions.APIException;
 4: import com.ecommerce.project.exceptions.ResourceNotFoundException;
 5: import com.ecommerce.project.model.Cart;
 6: import com.ecommerce.project.model.CartItem;
 7: import com.ecommerce.project.model.Product;
 8: import com.ecommerce.project.payload.CartDTO;
 9: import com.ecommerce.project.payload.ProductDTO;
10: import com.ecommerce.project.repositories.CartItemRepository;
11: import com.ecommerce.project.repositories.CartRepository;
12: import com.ecommerce.project.repositories.ProductRepository;
13: import com.ecommerce.project.util.AuthUtil;
14: import jakarta.transaction.Transactional;
15: import org.modelmapper.ModelMapper;
16: import org.springframework.beans.factory.annotation.Autowired;
17: import org.springframework.stereotype.Service;
18:
19: import java.util.List;
20: import java.util.stream.Collectors;
21: import java.util.stream.Stream;
22:
23: @Service
24: public class CartServiceImpl implements CartService{
25:     @Autowired
26:     private CartRepository cartRepository;
27:
28:     @Autowired
29:     private AuthUtil authUtil;
30:
31:     @Autowired
32:     ProductRepository productRepository;
33:
34:     @Autowired
35:     CartItemRepository cartItemRepository;
36:
37:     @Autowired
38:     ModelMapper modelMapper;
39:
40:     @Override
41:     public CartDTO addProductToCart(Long productId, Integer quantity) {
42:         Cart cart  = createCart();
43:
44:         Product product = productRepository.findById(productId)
45:                 .orElseThrow(() -> new ResourceNotFoundException("Product", "productId", produc
    tId));
46:
47:         CartItem cartItem = cartItemRepository.findCartItemByProductIdAndCartId(cart.getCartId(
    ), productId);
48:
49:         if (cartItem != null) {
50:             throw new APIException("Product " + product.getProductName() + " already exists in
    the cart");
51:         }
52:
53:         if (product.getQuantity() == 0) {
54:             throw new APIException(product.getProductName() + " is not available");
55:         }
56:
57:         if (product.getQuantity() < quantity) {
58:             throw new APIException("Please, make an order of the " + product.getProductName()
```

```
                    + " less than or equal to the quantity " + product.getQuantity() + ".");
        }

        CartItem newCartItem = new CartItem();

        newCartItem.setProduct(product);
        newCartItem.setCart(cart);
        newCartItem.setQuantity(quantity);
        newCartItem.setDiscount(product.getDiscount());
        newCartItem.setProductPrice(product.getSpecialPrice());

        cartItemRepository.save(newCartItem);

        product.setQuantity(product.getQuantity());

        cart.setTotalPrice(cart.getTotalPrice() + (product.getSpecialPrice() * quantity));

        cartRepository.save(cart);

        CartDTO cartDTO = modelMapper.map(cart, CartDTO.class);

        List<CartItem> cartItems = cart.getCartItems();

        Stream<ProductDTO> productStream = cartItems.stream().map(item -> {
            ProductDTO map = modelMapper.map(item.getProduct(), ProductDTO.class);
            map.setQuantity(item.getQuantity());
            return map;
        });

        cartDTO.setProducts(productStream.toList());

        return cartDTO;
    }

    @Override
    public List<CartDTO> getAllCarts() {
        List<Cart> carts = cartRepository.findAll();

        if (carts.size() == 0) {
            throw new APIException("No cart exists");
        }

        List<CartDTO> cartDTOs = carts.stream().map(cart -> {
            CartDTO cartDTO = modelMapper.map(cart, CartDTO.class);

            List<ProductDTO> products = cart.getCartItems().stream().map(cartItem -> {
                ProductDTO productDTO = modelMapper.map(cartItem.getProduct(), ProductDTO.class
);
                productDTO.setQuantity(cartItem.getQuantity()); // Set the quantity from CartIt
em
                return productDTO;
            }).collect(Collectors.toList());


            cartDTO.setProducts(products);

            return cartDTO;

        }).collect(Collectors.toList());

        return cartDTOs;
    }

    @Override
    public CartDTO getCart(String emailId, Long cartId) {
        Cart cart = cartRepository.findCartByEmailAndCartId(emailId, cartId);
        if (cart == null){
            throw new ResourceNotFoundException("Cart", "cartId", cartId);
        }
        CartDTO cartDTO = modelMapper.map(cart, CartDTO.class);
        cart.getCartItems().forEach(c ->
                c.getProduct().setQuantity(c.getQuantity()));
        List<ProductDTO> products = cart.getCartItems().stream()
                .map(p -> modelMapper.map(p.getProduct(), ProductDTO.class))
                .toList();
        cartDTO.setProducts(products);
        return cartDTO;
    }

    @Transactional
```

```
    @Override
    public CartDTO updateProductQuantityInCart(Long productId, Integer quantity) {

        String emailId = authUtil.loggedInEmail();
        Cart userCart = cartRepository.findCartByEmail(emailId);
        Long cartId  = userCart.getCartId();

        Cart cart = cartRepository.findById(cartId)
                .orElseThrow(() -> new ResourceNotFoundException("Cart", "cartId", cartId));

        Product product = productRepository.findById(productId)
                .orElseThrow(() -> new ResourceNotFoundException("Product", "productId", produc
    tId));

        if (product.getQuantity() == 0) {
            throw new APIException(product.getProductName() + " is not available");
        }

        if (product.getQuantity() < quantity) {
            throw new APIException("Please, make an order of the " + product.getProductName()
                    + " less than or equal to the quantity " + product.getQuantity() + ".");
        }

        CartItem cartItem = cartItemRepository.findCartItemByProductIdAndCartId(cartId, product
    Id);

        if (cartItem == null) {
            throw new APIException("Product " + product.getProductName() + " not available in t
    he cart!!!");
        }

        // Calculate new quantity
        int newQuantity = cartItem.getQuantity() + quantity;

        // Validation to prevent negative quantities
        if (newQuantity < 0) {
            throw new APIException("The resulting quantity cannot be negative.");
        }

        if (newQuantity == 0){
            deleteProductFromCart(cartId, productId);
        } else {
            cartItem.setProductPrice(product.getSpecialPrice());
            cartItem.setQuantity(cartItem.getQuantity() + quantity);
            cartItem.setDiscount(product.getDiscount());
            cart.setTotalPrice(cart.getTotalPrice() + (cartItem.getProductPrice() * quantity));
            cartRepository.save(cart);
        }

        CartItem updatedItem = cartItemRepository.save(cartItem);
        if(updatedItem.getQuantity() == 0){
            cartItemRepository.deleteById(updatedItem.getCartItemId());
        }


        CartDTO cartDTO = modelMapper.map(cart, CartDTO.class);

        List<CartItem> cartItems = cart.getCartItems();

        Stream<ProductDTO> productStream = cartItems.stream().map(item -> {
            ProductDTO prd = modelMapper.map(item.getProduct(), ProductDTO.class);
            prd.setQuantity(item.getQuantity());
            return prd;
        });


        cartDTO.setProducts(productStream.toList());

        return cartDTO;
    }


    private Cart createCart() {
        Cart userCart  = cartRepository.findCartByEmail(authUtil.loggedInEmail());
        if(userCart != null){
            return userCart;
        }

        Cart cart = new Cart();
        cart.setTotalPrice(0.00);
```

```
214:         cart.setUser(authUtil.loggedInUser());
215:         Cart newCart =  cartRepository.save(cart);
216:
217:         return newCart;
218:     }
219:
220:
221:     @Transactional
222:     @Override
223:     public String deleteProductFromCart(Long cartId, Long productId) {
224:         Cart cart = cartRepository.findById(cartId)
225:                 .orElseThrow(() -> new ResourceNotFoundException("Cart", "cartId", cartId));
226:
227:         CartItem cartItem = cartItemRepository.findCartItemByProductIdAndCartId(cartId, product
    Id);
228:
229:         if (cartItem == null) {
230:             throw new ResourceNotFoundException("Product", "productId", productId);
231:         }
232:
233:         cart.setTotalPrice(cart.getTotalPrice() -
234:                 (cartItem.getProductPrice() * cartItem.getQuantity()));
235:
236:         cartItemRepository.deleteCartItemByProductIdAndCartId(cartId, productId);
237:
238:         return "Product " + cartItem.getProduct().getProductName() + " removed from the cart !!
    !";
239:     }
240:
241:
242:     @Override
243:     public void updateProductInCarts(Long cartId, Long productId) {
244:         Cart cart = cartRepository.findById(cartId)
245:                 .orElseThrow(() -> new ResourceNotFoundException("Cart", "cartId", cartId));
246:
247:         Product product = productRepository.findById(productId)
248:                 .orElseThrow(() -> new ResourceNotFoundException("Product", "productId", produc
    tId));
249:
250:         CartItem cartItem = cartItemRepository.findCartItemByProductIdAndCartId(cartId, product
    Id);
251:
252:         if (cartItem == null) {
253:             throw new APIException("Product " + product.getProductName() + " not available in t
    he cart!!!");
254:         }
255:
256:         double cartPrice = cart.getTotalPrice()
257:                 - (cartItem.getProductPrice() * cartItem.getQuantity());
258:
259:         cartItem.setProductPrice(product.getSpecialPrice());
260:
261:         cart.setTotalPrice(cartPrice
262:                 + (cartItem.getProductPrice() * cartItem.getQuantity()));
263:
264:         cartItem = cartItemRepository.save(cartItem);
265:     }
266:
267: }
```

File Path: src/main/java/com/ecommerce/project/service/CategoryService.java

COMPONENT PROFILE

Stereotype:	**Service Interface**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'CategoryService' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.model.Category;
 4: import com.ecommerce.project.payload.CategoryDTO;
 5: import com.ecommerce.project.payload.CategoryResponse;
 6:
 7: public interface CategoryService {
 8:     CategoryResponse getAllCategories(Integer pageNumber, Integer pageSize, String sortBy, Stri
    ng sortOrder);
 9:     CategoryDTO createCategory(CategoryDTO categoryDTO);
10:
11:     CategoryDTO deleteCategory(Long categoryId);
12:
13:     CategoryDTO updateCategory(CategoryDTO categoryDTO, Long categoryId);
14: }
```

File Path: src/main/java/com/ecommerce/project/service/CategoryServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service*
Injected Dependencies:	categoryRepository (CategoryRepository), modelMapper (ModelMapper)

Technical Explanation of Code Logic:

Executes key transaction rules and calculations for 'Category'. It handles validation thresholds, queries database records, and mappings configurations.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.exceptions.APIException;
 4: import com.ecommerce.project.exceptions.ResourceNotFoundException;
 5: import com.ecommerce.project.model.Category;
 6: import com.ecommerce.project.payload.CategoryDTO;
 7: import com.ecommerce.project.payload.CategoryResponse;
 8: import com.ecommerce.project.repositories.CategoryRepository;
 9: import org.modelmapper.ModelMapper;
10: import org.springframework.beans.factory.annotation.Autowired;
11: import org.springframework.data.domain.Page;
12: import org.springframework.data.domain.PageRequest;
13: import org.springframework.data.domain.Pageable;
14: import org.springframework.data.domain.Sort;
15: import org.springframework.stereotype.Service;
16:
17: import java.util.List;
18:
19: @Service
20: public class CategoryServiceImpl implements CategoryService{
21:
22:     @Autowired
23:     private CategoryRepository categoryRepository;
24:
25:     @Autowired
26:     private ModelMapper modelMapper;
27:
28:     @Override
29:     public CategoryResponse getAllCategories(Integer pageNumber, Integer pageSize, String sortB
    y, String sortOrder) {
30:         Sort sortByAndOrder = sortOrder.equalsIgnoreCase("asc")
31:                 ? Sort.by(sortBy).ascending()
32:                 : Sort.by(sortBy).descending();
33:
34:         Pageable pageDetails = PageRequest.of(pageNumber, pageSize, sortByAndOrder);
35:         Page<Category> categoryPage = categoryRepository.findAll(pageDetails);
36:
37:         List<Category> categories = categoryPage.getContent();
38:         if (categories.isEmpty())
```

```
39:             throw new APIException("No category created till now.");
40:
41:         List<CategoryDTO> categoryDTOS = categories.stream()
42:                 .map(category -> modelMapper.map(category, CategoryDTO.class))
43:                 .toList();
44:
45:         CategoryResponse categoryResponse = new CategoryResponse();
46:         categoryResponse.setContent(categoryDTOS);
47:         categoryResponse.setPageNumber(categoryPage.getNumber());
48:         categoryResponse.setPageSize(categoryPage.getSize());
49:         categoryResponse.setTotalElements(categoryPage.getTotalElements());
50:         categoryResponse.setTotalPages(categoryPage.getTotalPages());
51:         categoryResponse.setLastPage(categoryPage.isLast());
52:         return categoryResponse;
53:     }
54:
55:     @Override
56:     public CategoryDTO createCategory(CategoryDTO categoryDTO) {
57:         Category category = modelMapper.map(categoryDTO, Category.class);
58:         Category categoryFromDb = categoryRepository.findByCategoryName(category.getCategoryNam
    e());
59:         if (categoryFromDb != null)
60:             throw new APIException("Category with the name " + category.getCategoryName() + " a
    lready exists !!!");
61:         Category savedCategory = categoryRepository.save(category);
62:         return modelMapper.map(savedCategory, CategoryDTO.class);
63:     }
64:
65:     @Override
66:     public CategoryDTO deleteCategory(Long categoryId) {
67:         Category category = categoryRepository.findById(categoryId)
68:                 .orElseThrow(() -> new ResourceNotFoundException("Category","categoryId",catego
    ryId));
69:
70:         categoryRepository.delete(category);
71:         return modelMapper.map(category, CategoryDTO.class);
72:     }
73:
74:     @Override
75:     public CategoryDTO updateCategory(CategoryDTO categoryDTO, Long categoryId) {
76:         Category savedCategory = categoryRepository.findById(categoryId)
77:                 .orElseThrow(() -> new ResourceNotFoundException("Category","categoryId",catego
    ryId));
78:
79:         Category category = modelMapper.map(categoryDTO, Category.class);
80:         category.setCategoryId(categoryId);
81:         savedCategory = categoryRepository.save(category);
82:         return modelMapper.map(savedCategory, CategoryDTO.class);
83:     }
84: }
```

File Path: src/main/java/com/ecommerce/project/service/FileService.java

COMPONENT PROFILE

Stereotype: **Service Interface**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'FileService' workflow executing smoothly.

Source Code Listing:

```
1: package com.ecommerce.project.service;
2:
3: import org.springframework.web.multipart.MultipartFile;
4:
5: import java.io.IOException;
6:
7: public interface FileService {
8:     String uploadImage(String path, MultipartFile file) throws IOException;
9: }
```

File Path: src/main/java/com/ecommerce/project/service/FileServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service*

Technical Explanation of Code Logic:

Executes key transaction rules and calculations for 'File'. It handles validation thresholds, queries database records, and mappings configurations.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import org.springframework.stereotype.Service;
 4: import org.springframework.web.multipart.MultipartFile;
 5:
 6: import java.io.File;
 7: import java.io.IOException;
 8: import java.nio.file.Files;
 9: import java.nio.file.Paths;
10: import java.util.UUID;
11:
12: @Service
13: public class FileServiceImpl implements FileService {
14:
15:     @Override
16:     public String uploadImage(String path, MultipartFile file) throws IOException {
17:         String originalFileName = file.getOriginalFilename();
18:         String randomId = UUID.randomUUID().toString();
19:         String fileName = randomId.concat(originalFileName.substring(originalFileName.lastIndex
    Of('.')));
20:         String filePath = path + File.separator + fileName;
21:
22:         File folder = new File(path);
23:         if (!folder.exists())
24:             folder.mkdir();
25:
26:         Files.copy(file.getInputStream(), Paths.get(filePath));
27:         return fileName;
28:     }
29: }
```

File Path: src/main/java/com/ecommerce/project/service/OrderService.java

COMPONENT PROFILE

Stereotype:	**Service Interface**
Active Annotations:	*@Transactional*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'OrderService' workflow executing smoothly.

Source Code Listing:

```
1: package com.ecommerce.project.service;
2:
3: import com.ecommerce.project.payload.OrderDTO;
4: import jakarta.transaction.Transactional;
5:
6: public interface OrderService {
7:     @Transactional
8:     OrderDTO placeOrder(String emailId, Long addressId, String paymentMethod, String pgName, St
   ring pgPaymentId, String pgStatus, String pgResponseMessage);
9: }
```

File Path: src/main/java/com/ecommerce/project/service/OrderServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service, @Transactional*

Technical Explanation of Code Logic:

OrderServiceImpl coordinates checkout actions. It loads the customer's cart, verifies items, deducts quantities from product stock, initializes order/payment records, clears the cart, and returns detailed DTO structures.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.exceptions.APIException;
 4: import com.ecommerce.project.exceptions.ResourceNotFoundException;
 5: import com.ecommerce.project.model.*;
 6: import com.ecommerce.project.payload.OrderDTO;
 7: import com.ecommerce.project.payload.OrderItemDTO;
 8: import com.ecommerce.project.repositories.*;
 9: import jakarta.transaction.Transactional;
10: import org.modelmapper.ModelMapper;
11: import org.springframework.beans.factory.annotation.Autowired;
12: import org.springframework.stereotype.Service;
13:
14: import java.time.LocalDate;
15: import java.util.ArrayList;
16: import java.util.List;
17:
18: @Service
19: public class OrderServiceImpl implements OrderService {
20:
21:     @Autowired
22:     CartRepository cartRepository;
23:
24:     @Autowired
25:     AddressRepository addressRepository;
26:
27:     @Autowired
28:     OrderItemRepository orderItemRepository;
29:
30:     @Autowired
31:     OrderRepository orderRepository;
32:
33:     @Autowired
34:     PaymentRepository paymentRepository;
35:
36:     @Autowired
37:     CartService cartService;
38:
39:     @Autowired
40:     ModelMapper modelMapper;
41:
42:     @Autowired
43:     ProductRepository productRepository;
44:
45:     @Override
46:     @Transactional
47:     public OrderDTO placeOrder(String emailId, Long addressId, String paymentMethod, String pgN
    ame, String pgPaymentId, String pgStatus, String pgResponseMessage) {
48:         Cart cart = cartRepository.findCartByEmail(emailId);
49:         if (cart == null) {
50:             throw new ResourceNotFoundException("Cart", "email", emailId);
51:         }
52:
53:         Address address = addressRepository.findById(addressId)
54:                 .orElseThrow(() -> new ResourceNotFoundException("Address", "addressId", addres
    sId));
55:
56:         Order order = new Order();
57:         order.setEmail(emailId);
58:         order.setOrderDate(LocalDate.now());
59:         order.setTotalAmount(cart.getTotalPrice());
60:         order.setOrderStatus("Order Accepted !");
61:         order.setAddress(address);
```

```
 62:
 63:         Payment payment = new Payment(paymentMethod, pgPaymentId, pgStatus, pgResponseMessage,
    pgName);
 64:         payment.setOrder(order);
 65:         payment = paymentRepository.save(payment);
 66:         order.setPayment(payment);
 67:
 68:         Order savedOrder = orderRepository.save(order);
 69:
 70:         List<CartItem> cartItems = cart.getCartItems();
 71:         if (cartItems.isEmpty()) {
 72:             throw new APIException("Cart is empty");
 73:         }
 74:
 75:         List<OrderItem> orderItems = new ArrayList<>();
 76:         for (CartItem cartItem : cartItems) {
 77:             OrderItem orderItem = new OrderItem();
 78:             orderItem.setProduct(cartItem.getProduct());
 79:             orderItem.setQuantity(cartItem.getQuantity());
 80:             orderItem.setDiscount(cartItem.getDiscount());
 81:             orderItem.setOrderedProductPrice(cartItem.getProductPrice());
 82:             orderItem.setOrder(savedOrder);
 83:             orderItems.add(orderItem);
 84:         }
 85:
 86:         orderItems = orderItemRepository.saveAll(orderItems);
 87:
 88:         cart.getCartItems().forEach(item -> {
 89:             int quantity = item.getQuantity();
 90:             Product product = item.getProduct();
 91:
 92:             // Reduce stock quantity
 93:             product.setQuantity(product.getQuantity() - quantity);
 94:
 95:             // Save product back to the database
 96:             productRepository.save(product);
 97:
 98:             // Remove items from cart
 99:             cartService.deleteProductFromCart(cart.getCartId(), item.getProduct().getProductId(
    ));
100:         });
101:
102:         OrderDTO orderDTO = modelMapper.map(savedOrder, OrderDTO.class);
103:         orderItems.forEach(item -> orderDTO.getOrderItems().add(modelMapper.map(item, OrderItem
    DTO.class)));
104:
105:         orderDTO.setAddressId(addressId);
106:
107:         return orderDTO;
108:     }
109: }
```

File Path: src/main/java/com/ecommerce/project/service/ProductService.java

COMPONENT PROFILE

Stereotype: **Service Interface**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'ProductService' workflow executing smoothly.

Source Code Listing:

```
1: package com.ecommerce.project.service;
2:
3: import com.ecommerce.project.payload.ProductDTO;
4: import com.ecommerce.project.payload.ProductResponse;
5: import org.springframework.web.multipart.MultipartFile;
6:
7: import java.io.IOException;
8:
```

```
 9: public interface ProductService {
10:     ProductDTO addProduct(Long categoryId, ProductDTO product);
11:
12:     ProductResponse getAllProducts(Integer pageNumber, Integer pageSize, String sortBy, String
    sortOrder);
13:
14:     ProductResponse searchByCategory(Long categoryId, Integer pageNumber, Integer pageSize, Str
    ing sortBy, String sortOrder);
15:
16:     ProductResponse searchProductByKeyword(String keyword, Integer pageNumber, Integer pageSize
    , String sortBy, String sortOrder);
17:
18:     ProductDTO updateProduct(Long productId, ProductDTO product);
19:
20:     ProductDTO deleteProduct(Long productId);
21:
22:     ProductDTO updateProductImage(Long productId, MultipartFile image) throws IOException;
23: }
```

File Path: src/main/java/com/ecommerce/project/service/ProductServiceImpl.java

COMPONENT PROFILE

Stereotype:	**Business Service Implementation**
Active Annotations:	*@Service, @Value*
Injected Dependencies:	cartRepository (CartRepository), cartService (CartService), productRepository (ProductRepository), categoryRepository

Technical Explanation of Code Logic:

ProductServiceImpl executes product searches, updates, and uploads. It handles pagination, category association, image file management (saving uploaded images to disk and serving them back), and inventory updates.

Source Code Listing:

```
 1: package com.ecommerce.project.service;
 2:
 3: import com.ecommerce.project.exceptions.APIException;
 4: import com.ecommerce.project.exceptions.ResourceNotFoundException;
 5: import com.ecommerce.project.model.Cart;
 6: import com.ecommerce.project.model.Category;
 7: import com.ecommerce.project.model.Product;
 8: import com.ecommerce.project.payload.CartDTO;
 9: import com.ecommerce.project.payload.ProductDTO;
10: import com.ecommerce.project.payload.ProductResponse;
11: import com.ecommerce.project.repositories.CartRepository;
12: import com.ecommerce.project.repositories.CategoryRepository;
13: import com.ecommerce.project.repositories.ProductRepository;
14: import org.modelmapper.ModelMapper;
15: import org.springframework.beans.factory.annotation.Autowired;
16: import org.springframework.beans.factory.annotation.Value;
17: import org.springframework.data.domain.Page;
18: import org.springframework.data.domain.PageRequest;
19: import org.springframework.data.domain.Pageable;
20: import org.springframework.data.domain.Sort;
21: import org.springframework.stereotype.Service;
22: import org.springframework.web.multipart.MultipartFile;
23:
24: import java.io.IOException;
25: import java.util.List;
26: import java.util.stream.Collectors;
27:
28: @Service
29: public class ProductServiceImpl implements ProductService {
30:     @Autowired
31:     private CartRepository cartRepository;
32:
33:     @Autowired
34:     private CartService cartService;
35:
36:     @Autowired
```

```
37:     private ProductRepository productRepository;
38:
39:     @Autowired
40:     private CategoryRepository categoryRepository;
41:
42:     @Autowired
43:     private ModelMapper modelMapper;
44:
45:     @Autowired
46:     private FileService fileService;
47:
48:     @Value("${project.image}")
49:     private String path;
50:
51:     @Override
52:     public ProductDTO addProduct(Long categoryId, ProductDTO productDTO) {
53:         Category category = categoryRepository.findById(categoryId)
54:                 .orElseThrow(() ->
55:                         new ResourceNotFoundException("Category", "categoryId", categoryId));
56:
57:         boolean isProductNotPresent = true;
58:
59:         List<Product> products = category.getProducts();
60:         for (Product value : products) {
61:             if (value.getProductName().equals(productDTO.getProductName())) {
62:                 isProductNotPresent = false;
63:                 break;
64:             }
65:         }
66:
67:         if (isProductNotPresent) {
68:             Product product = modelMapper.map(productDTO, Product.class);
69:             product.setImage("default.png");
70:             product.setCategory(category);
71:             double specialPrice = product.getPrice() -
72:                     ((product.getDiscount() * 0.01) * product.getPrice());
73:             product.setSpecialPrice(specialPrice);
74:             Product savedProduct = productRepository.save(product);
75:             return modelMapper.map(savedProduct, ProductDTO.class);
76:         } else {
77:             throw new APIException("Product already exist!!");
78:         }
79:     }
80:
81:     @Override
82:     public ProductResponse getAllProducts(Integer pageNumber, Integer pageSize, String sortBy,
    String sortOrder) {
83:         Sort sortByAndOrder = sortOrder.equalsIgnoreCase("asc")
84:                 ? Sort.by(sortBy).ascending()
85:                 : Sort.by(sortBy).descending();
86:
87:         Pageable pageDetails = PageRequest.of(pageNumber, pageSize, sortByAndOrder);
88:         Page<Product> pageProducts = productRepository.findAll(pageDetails);
89:
90:         List<Product> products = pageProducts.getContent();
91:
92:         List<ProductDTO> productDTOS = products.stream()
93:                 .map(product -> modelMapper.map(product, ProductDTO.class))
94:                 .toList();
95:
96:         ProductResponse productResponse = new ProductResponse();
97:         productResponse.setContent(productDTOS);
98:         productResponse.setPageNumber(pageProducts.getNumber());
99:         productResponse.setPageSize(pageProducts.getSize());
100:        productResponse.setTotalElements(pageProducts.getTotalElements());
101:        productResponse.setTotalPages(pageProducts.getTotalPages());
102:        productResponse.setLastPage(pageProducts.isLast());
103:        return productResponse;
104:    }
105:
106:    @Override
107:    public ProductResponse searchByCategory(Long categoryId, Integer pageNumber, Integer pageSi
    ze, String sortBy, String sortOrder) {
108:        Category category = categoryRepository.findById(categoryId)
109:                .orElseThrow(() ->
110:                        new ResourceNotFoundException("Category", "categoryId", categoryId));
111:
112:        Sort sortByAndOrder = sortOrder.equalsIgnoreCase("asc")
113:                ? Sort.by(sortBy).ascending()
114:                : Sort.by(sortBy).descending();
```

```
115:
116:         Pageable pageDetails = PageRequest.of(pageNumber, pageSize, sortByAndOrder);
117:         Page<Product> pageProducts = productRepository.findByCategoryOrderByPriceAsc(category,
     pageDetails);
118:
119:         List<Product> products = pageProducts.getContent();
120:
121:         if(products.isEmpty()){
122:             throw new APIException(category.getCategoryName() + " category does not have any pr
     oducts");
123:         }
124:
125:         List<ProductDTO> productDTOS = products.stream()
126:                 .map(product -> modelMapper.map(product, ProductDTO.class))
127:                 .toList();
128:
129:         ProductResponse productResponse = new ProductResponse();
130:         productResponse.setContent(productDTOS);
131:         productResponse.setPageNumber(pageProducts.getNumber());
132:         productResponse.setPageSize(pageProducts.getSize());
133:         productResponse.setTotalElements(pageProducts.getTotalElements());
134:         productResponse.setTotalPages(pageProducts.getTotalPages());
135:         productResponse.setLastPage(pageProducts.isLast());
136:         return productResponse;
137:     }
138:
139:     @Override
140:     public ProductResponse searchProductByKeyword(String keyword, Integer pageNumber, Integer p
     ageSize, String sortBy, String sortOrder) {
141:         Sort sortByAndOrder = sortOrder.equalsIgnoreCase("asc")
142:                 ? Sort.by(sortBy).ascending()
143:                 : Sort.by(sortBy).descending();
144:
145:         Pageable pageDetails = PageRequest.of(pageNumber, pageSize, sortByAndOrder);
146:         Page<Product> pageProducts = productRepository.findByProductNameLikeIgnoreCase('%' + ke
     yword + '%', pageDetails);
147:
148:         List<Product> products = pageProducts.getContent();
149:         List<ProductDTO> productDTOS = products.stream()
150:                 .map(product -> modelMapper.map(product, ProductDTO.class))
151:                 .toList();
152:
153:         if(products.isEmpty()){
154:             throw new APIException("Products not found with keyword: " + keyword);
155:         }
156:
157:         ProductResponse productResponse = new ProductResponse();
158:         productResponse.setContent(productDTOS);
159:         productResponse.setPageNumber(pageProducts.getNumber());
160:         productResponse.setPageSize(pageProducts.getSize());
161:         productResponse.setTotalElements(pageProducts.getTotalElements());
162:         productResponse.setTotalPages(pageProducts.getTotalPages());
163:         productResponse.setLastPage(pageProducts.isLast());
164:         return productResponse;
165:     }
166:
167:     @Override
168:     public ProductDTO updateProduct(Long productId, ProductDTO productDTO) {
169:         Product productFromDb = productRepository.findById(productId)
170:                 .orElseThrow(() -> new ResourceNotFoundException("Product", "productId", produc
     tId));
171:
172:         Product product = modelMapper.map(productDTO, Product.class);
173:
174:         productFromDb.setProductName(product.getProductName());
175:         productFromDb.setDescription(product.getDescription());
176:         productFromDb.setQuantity(product.getQuantity());
177:         productFromDb.setDiscount(product.getDiscount());
178:         productFromDb.setPrice(product.getPrice());
179:         productFromDb.setSpecialPrice(product.getSpecialPrice());
180:
181:         Product savedProduct = productRepository.save(productFromDb);
182:
183:         List<Cart> carts = cartRepository.findCartsByProductId(productId);
184:
185:         List<CartDTO> cartDTOs = carts.stream().map(cart -> {
186:             CartDTO cartDTO = modelMapper.map(cart, CartDTO.class);
187:
188:             List<ProductDTO> products = cart.getCartItems().stream()
189:                     .map(p -> modelMapper.map(p.getProduct(), ProductDTO.class)).collect(Collec
```

```
    tors.toList());
190:
191:             cartDTO.setProducts(products);
192:
193:             return cartDTO;
194:
195:         }).collect(Collectors.toList());
196:
197:         cartDTOs.forEach(cart -> cartService.updateProductInCarts(cart.getCartId(), productId))
    ;
198:
199:         return modelMapper.map(savedProduct, ProductDTO.class);
200:     }
201:
202:     @Override
203:     public ProductDTO deleteProduct(Long productId) {
204:         Product product = productRepository.findById(productId)
205:                 .orElseThrow(() -> new ResourceNotFoundException("Product", "productId", produc
    tId));
206:
207:         // DELETE
208:         List<Cart> carts = cartRepository.findCartsByProductId(productId);
209:         carts.forEach(cart -> cartService.deleteProductFromCart(cart.getCartId(), productId));
210:
211:         productRepository.delete(product);
212:         return modelMapper.map(product, ProductDTO.class);
213:     }
214:
215:     @Override
216:     public ProductDTO updateProductImage(Long productId, MultipartFile image) throws IOExceptio
    n {
217:         Product productFromDb = productRepository.findById(productId)
218:                 .orElseThrow(() -> new ResourceNotFoundException("Product", "productId", produc
    tId));
219:
220:         String fileName = fileService.uploadImage(path, image);
221:         productFromDb.setImage(fileName);
222:
223:         Product updatedProduct = productRepository.save(productFromDb);
224:         return modelMapper.map(updatedProduct, ProductDTO.class);
225:     }
226:
227:
228: }
```

File Path: src/main/java/com/ecommerce/project/util/AuthUtil.java

COMPONENT PROFILE

Stereotype:	**Java Helper Component**
Active Annotations:	*@Component*

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'AuthUtil' workflow executing smoothly.

Source Code Listing:

```
 1: package com.ecommerce.project.util;
 2:
 3: import com.ecommerce.project.model.User;
 4: import com.ecommerce.project.repositories.UserRepository;
 5: import org.springframework.beans.factory.annotation.Autowired;
 6: import org.springframework.security.core.Authentication;
 7: import org.springframework.security.core.context.SecurityContextHolder;
 8: import org.springframework.security.core.userdetails.UsernameNotFoundException;
 9: import org.springframework.stereotype.Component;
10:
11: @Component
12: public class AuthUtil {
13:
14:     @Autowired
```

```
15:     UserRepository userRepository;
16:
17:     public String loggedInEmail(){
18:         Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
19:         User user = userRepository.findByUserName(authentication.getName())
20:                 .orElseThrow(() -> new UsernameNotFoundException("User Not Found with username:
     " + authentication.getName()));
21:
22:         return user.getEmail();
23:     }
24:
25:     public Long loggedInUserId(){
26:         Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
27:         User user = userRepository.findByUserName(authentication.getName())
28:                 .orElseThrow(() -> new UsernameNotFoundException("User Not Found with username:
     " + authentication.getName()));
29:
30:         return user.getUserId();
31:     }
32:
33:     public User loggedInUser(){
34:         Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
35:
36:         User user = userRepository.findByUserName(authentication.getName())
37:                 .orElseThrow(() -> new UsernameNotFoundException("User Not Found with username:
     " + authentication.getName()));
38:         return user;
39:
40:     }
41:
42:
43: }
```

File Path: src/main/resources/application.properties

COMPONENT PROFILE

Stereotype: **Java Helper Component**

Technical Explanation of Code Logic:

Provides logic, classes, settings, configurations or validation methods to keep the 'application.properties' workflow executing smoothly.

Source Code Listing:

```
 1: spring.application.name=sb-ecom
 2:
 3: spring.datasource.url=jdbc:postgresql://localhost:5432/myapp
 4: spring.datasource.username=postgres
 5: spring.datasource.password=Manohar2909
 6:
 7: spring.jpa.hibernate.ddl-auto=update
 8: spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
 9:
10: #spring.h2.console.enabled=true
11: #spring.datasource.url=jdbc:h2:mem:test
12:
13: project.image=images/
14:
15: #spring.jpa.show-sql=true
16: #spring.jpa.properties.hibernate.format_sql=true
17:
18:
19: spring.app.jwtSecret=mySecretKey123912738aopsgjnspkmndfsopkvajoirjg94gf2opfng2moknm
20: spring.app.jwtExpirationMs=3000000
21: spring.ecom.app.jwtCookieName=springBootEcom
22: server.port=5000
23:
24: #logging.level.org.springframework=DEBUG
25: #logging.level.org.hibernate.SQL=DEBUG
26: #logging.level.org.springframework.security=DEBUG
```

```
27: #logging.level.com.ecommerce.project=DEBUG
```